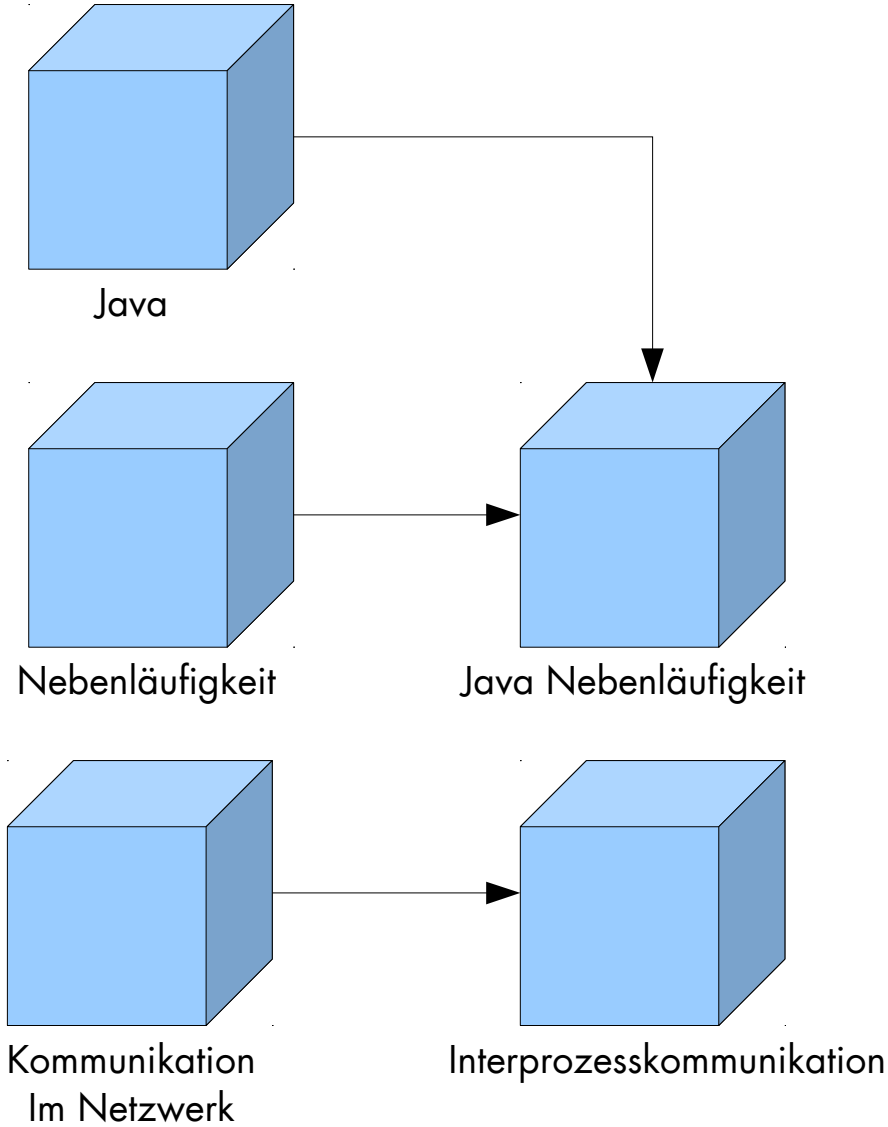
UNIVERSITÄT LEIPZIG
Institut für Wirtschaftsinformatik

Entwicklung verteilter Anwendungen

Interprozesskommunikation in Java

Pascal Kovacs

5. Mai 2020


Java
Nebenläufigkeit
Java Nebenläufigkeit
Kommunikation
Im Netzwerk
Interprozesskommunikation

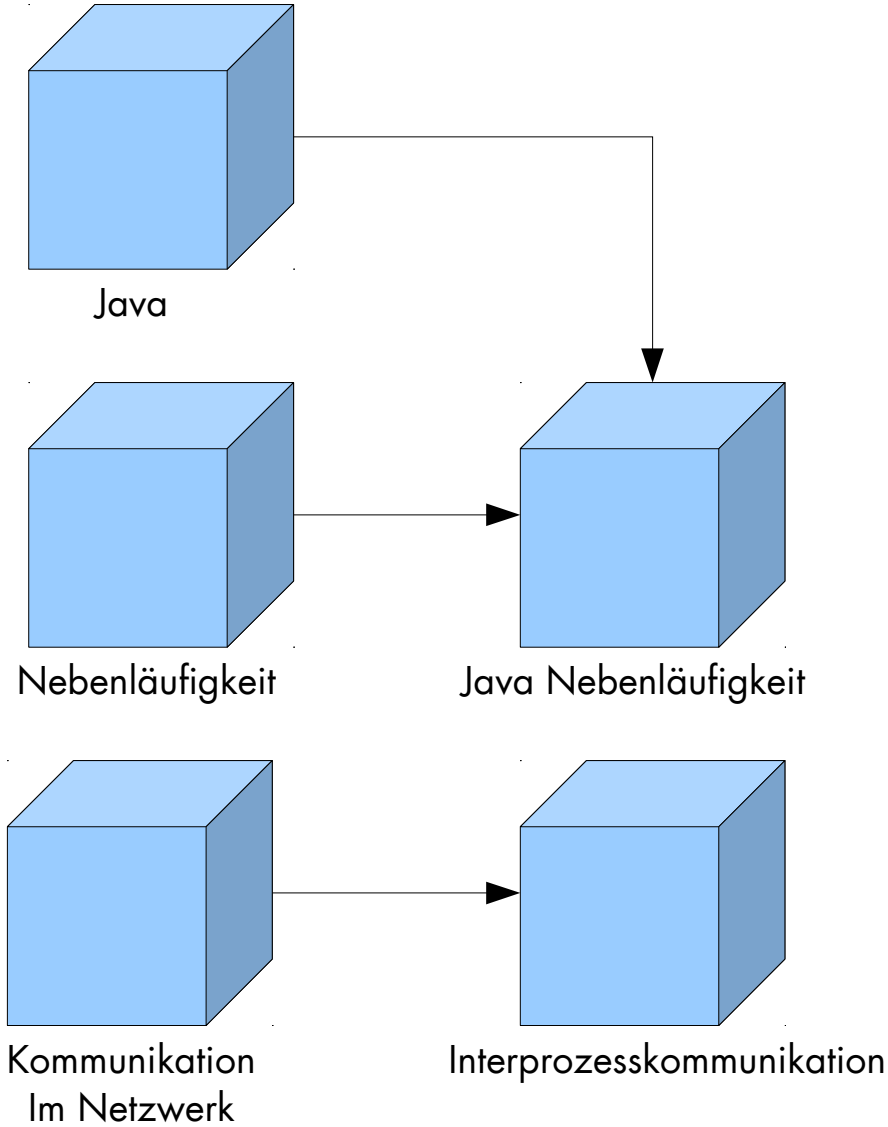
Ziele Interprozesskommunikation

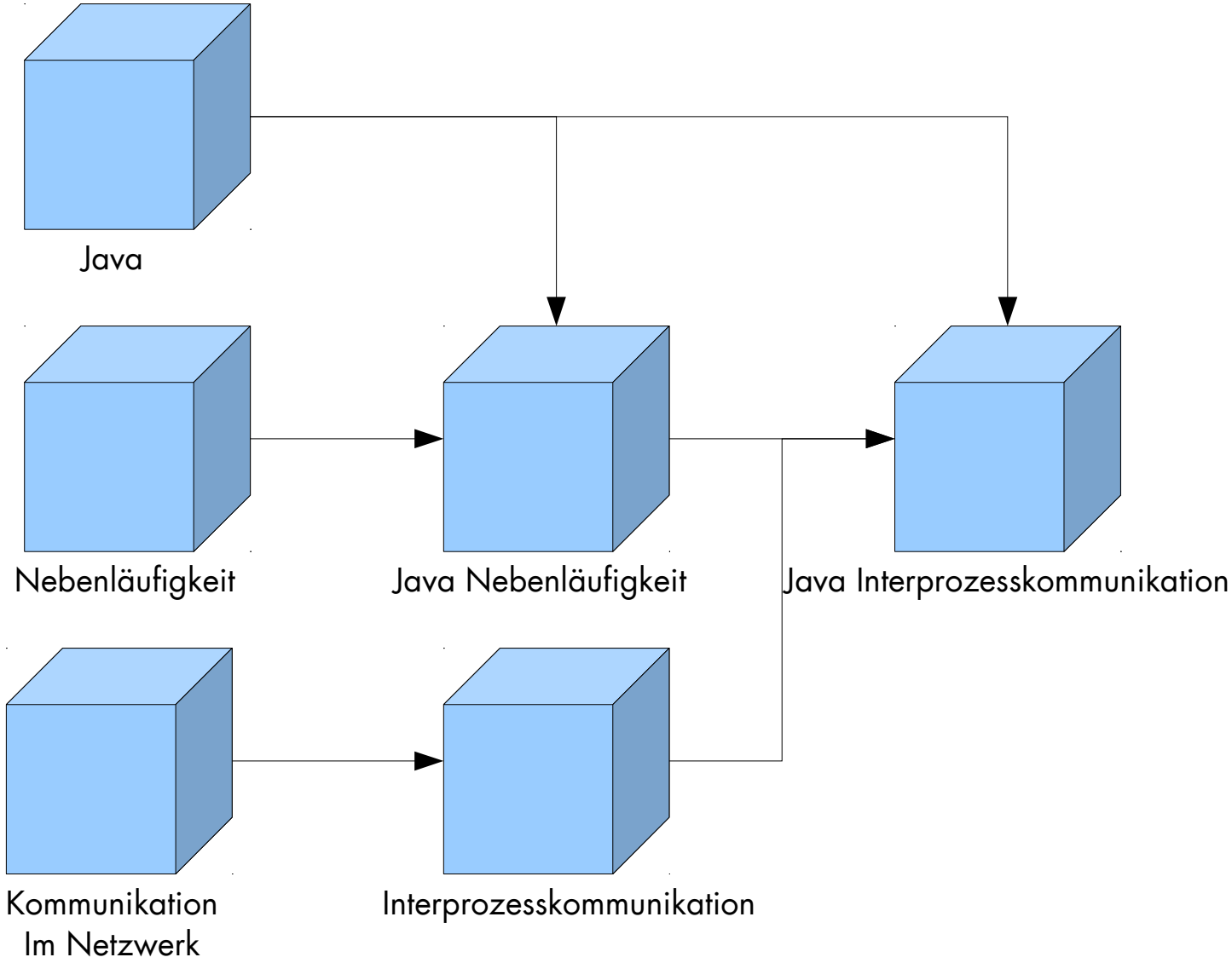
- IPK und Methoden verstehen
- IP und Ports
- TCP
- UDP
- Theorie Sockets

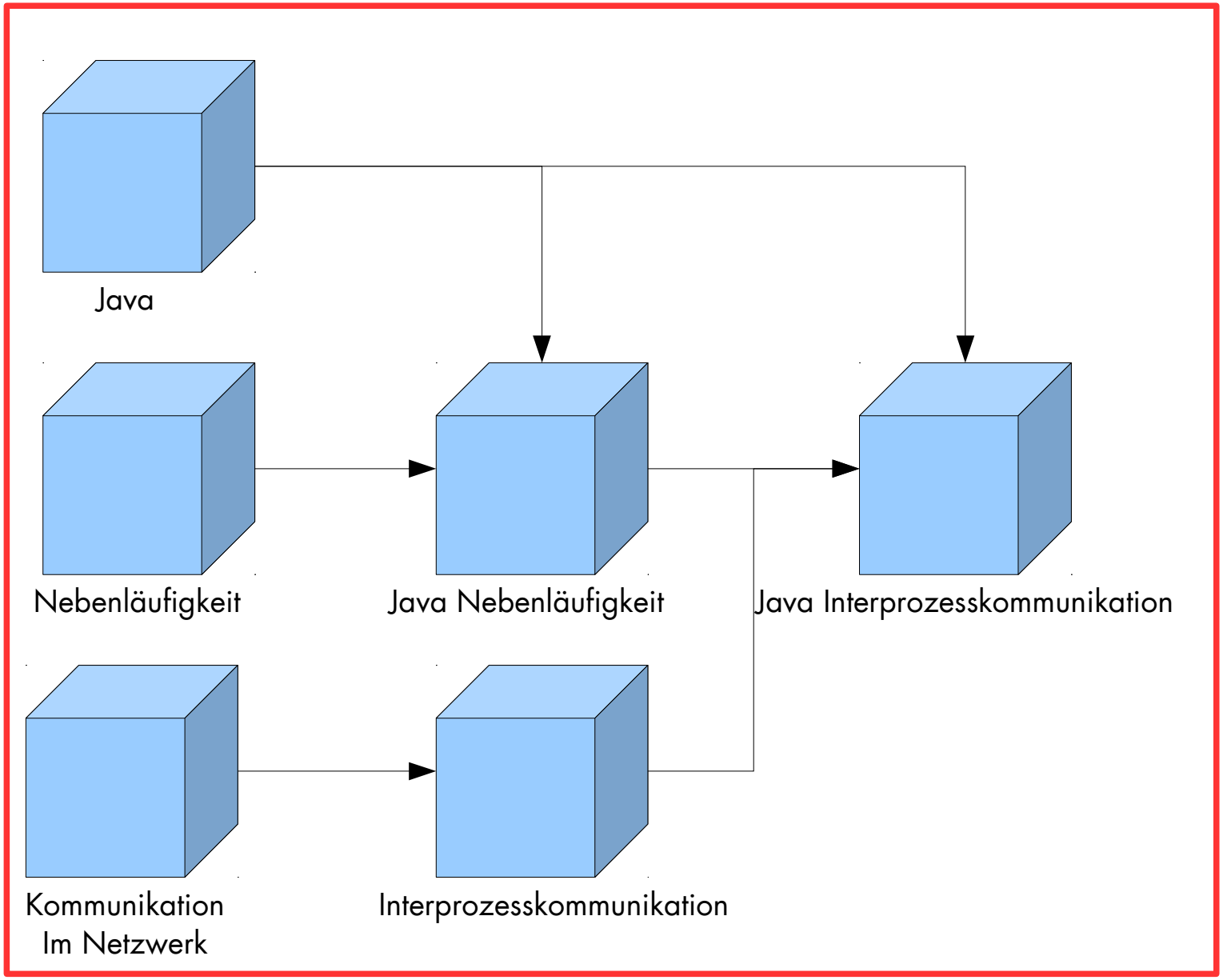
Kontrollfragen Interprozesskommunikation

- Welche Methoden zur IPK gibt es?
- Wie hängen Netzwerkkommunikation und Interprozesskommunikation zusammen?
- Was bestimmen IP-Adressen?
- Was bestimmen Ports?
- Was ist der zentrale Unterschied zwischen TCP und UDP?
- Welche Eigenschaften ergeben sich daraus für TCP und UDP?
- Zu welchen Schichten des ISO/OSI Referenzmodells gehören IP, TCP und UDP?
- Wie viele Sockets braucht es für ein- und ausgehenden IPK von zwei Teilnehmern?
- Wie funktionieren TCP und UDP mittels Sockets?

© 2020 Pascal Kovacs


Java
Nebenläufigkeit
Java Nebenläufigkeit
Kommunikation
Im Netzwerk
Interprozesskommunikation


Java
Nebenläufigkeit
Java Nebenläufigkeit
Java Interprozesskommunikation
Kommunikation
Im Netzwerk
Interprozesskommunikation


Java
Nebenläufigkeit
Java Nebenläufigkeit
Java Interprozesskommunikation
Kommunikation
Im Netzwerk
Interprozesskommunikation

Ziele Java Interprozesskommunikation

- Sockets in Java verstehen und Code lesen können
- Java RMI verstehen und die Unterschiede zu Sockets kennen

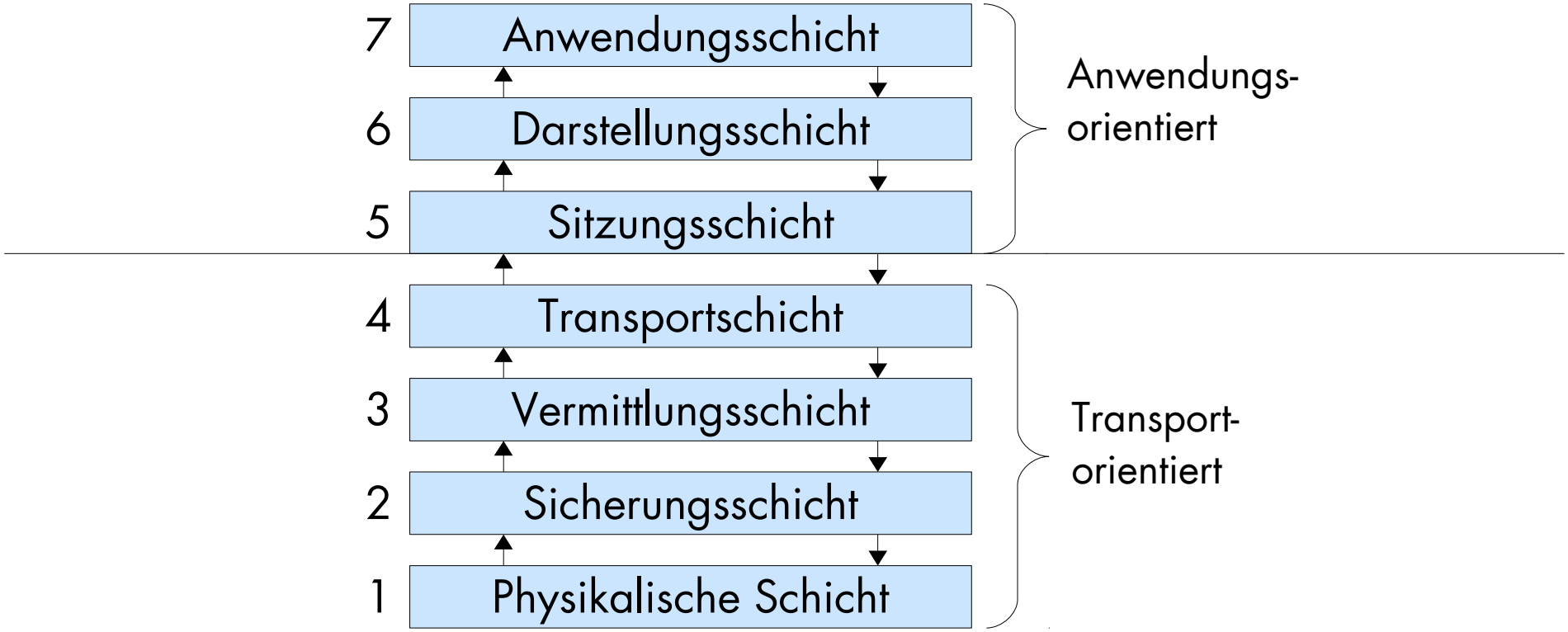
Kontrollfragen Java Interprozesskommunikation

- Welche Unterschiede gibt es bei der Implementierung einer IPK über TCP und UDP Sockets in Java?
- Welche Vorteile hat RMI im Vergleich zur IPK mittels Sockets?
- Wie ist die Architektur von RMI?
- Was ist der Unterschied zwischen Vererbung und dem Implementieren von Interfaces in Java?

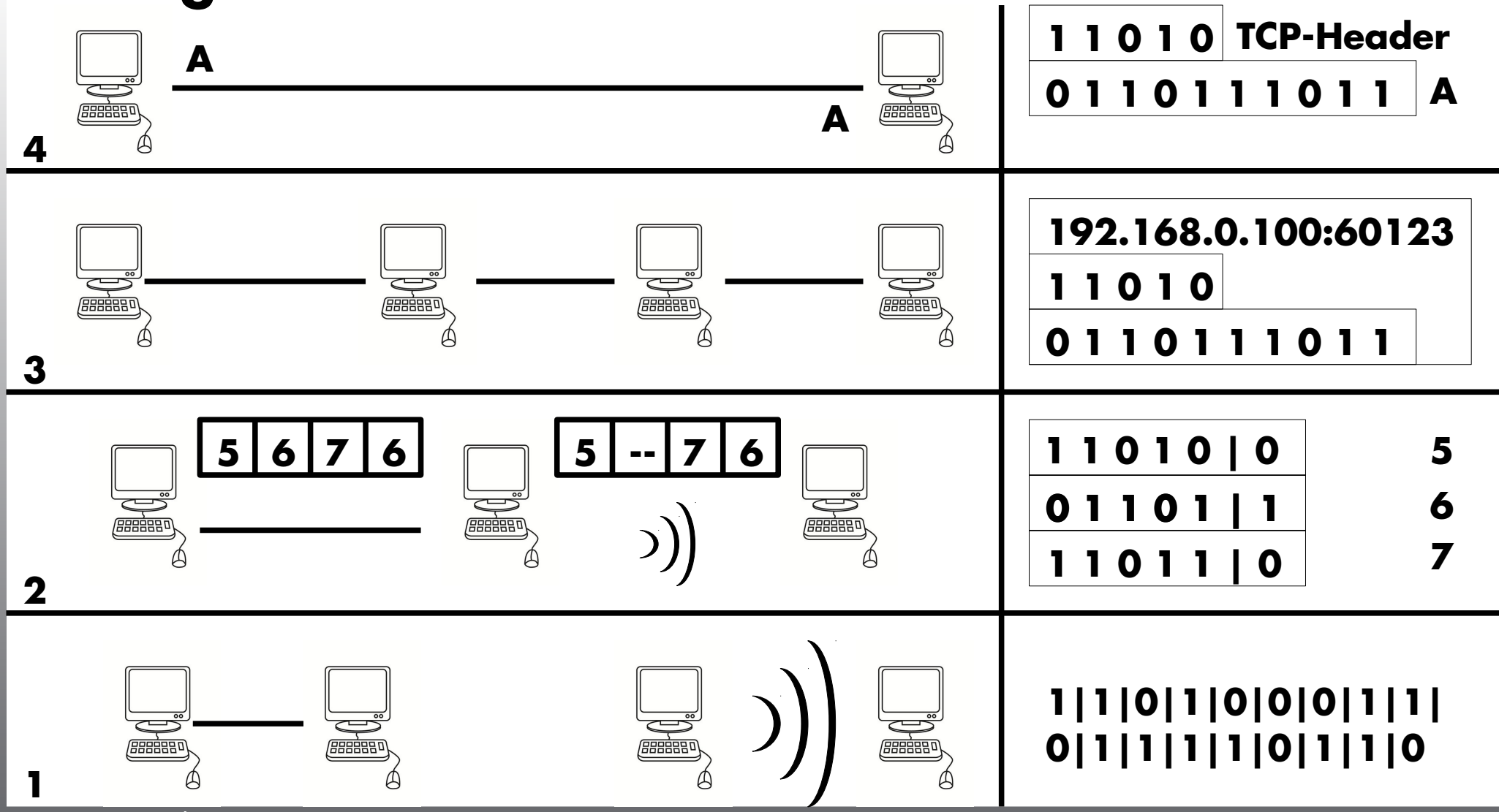
Übersicht

- Wiederholung
- TCP-Sockets
- UDP-Sockets
- Remote Method Invocation (RMI)

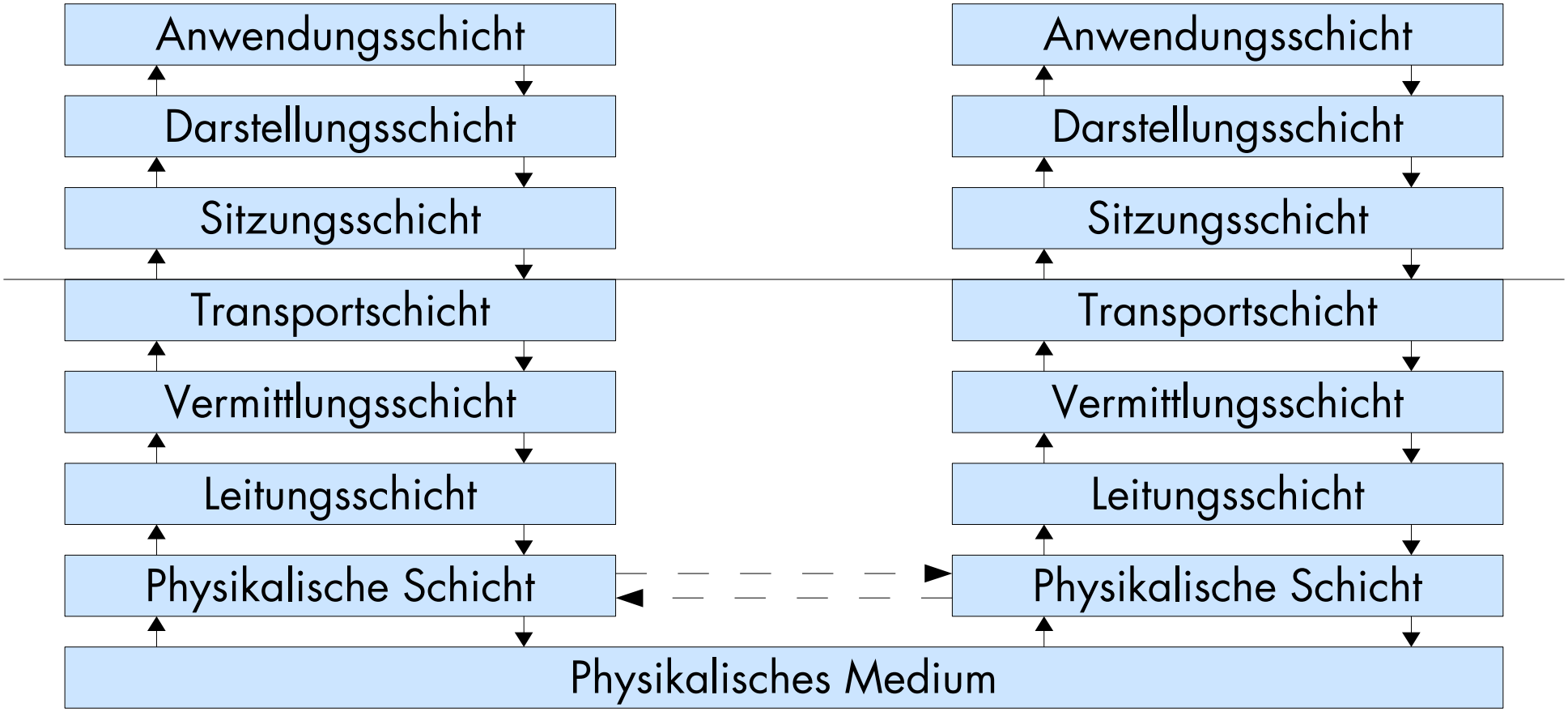
ISO/OSI-Referenzmodell



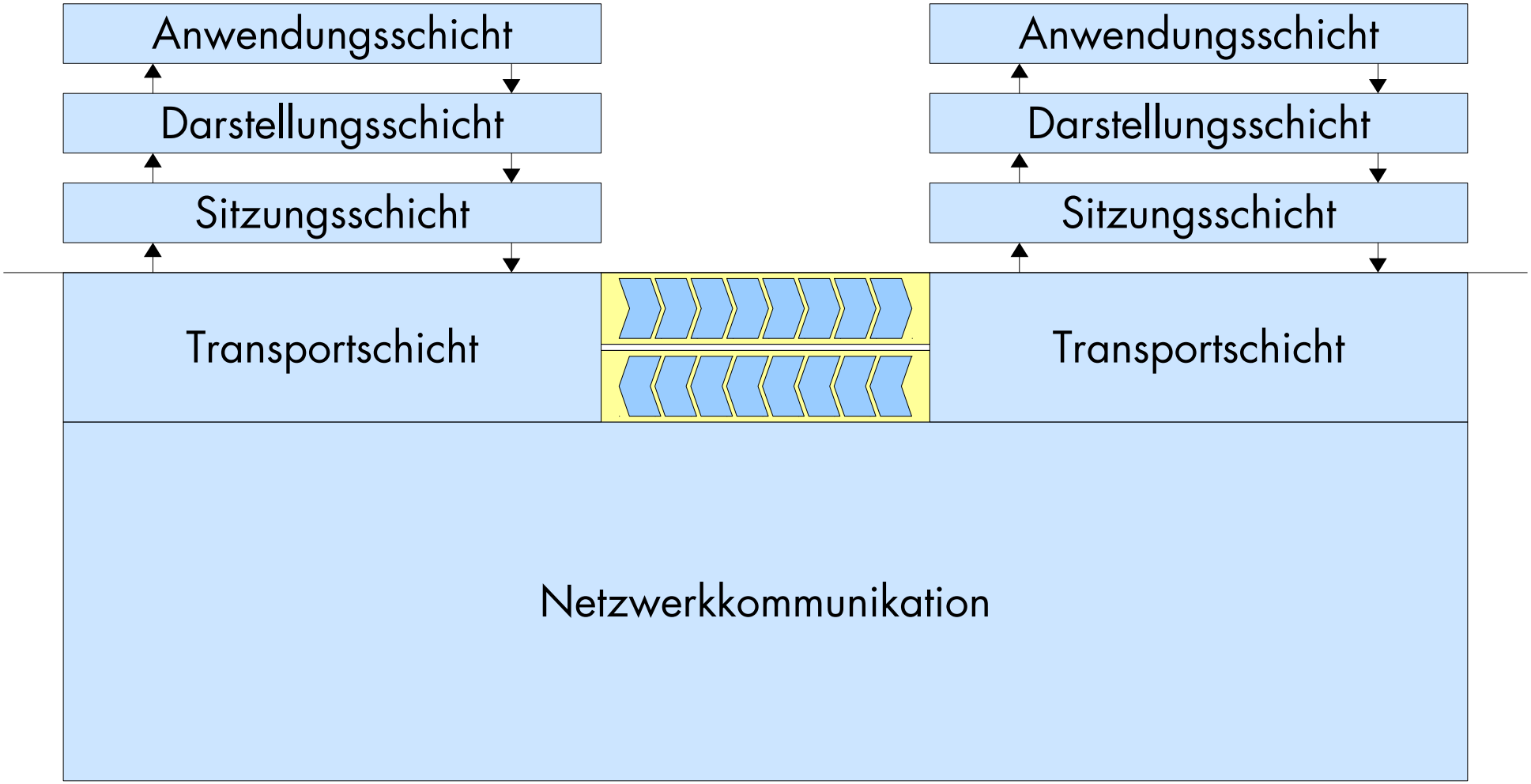
Logische Kommunikation



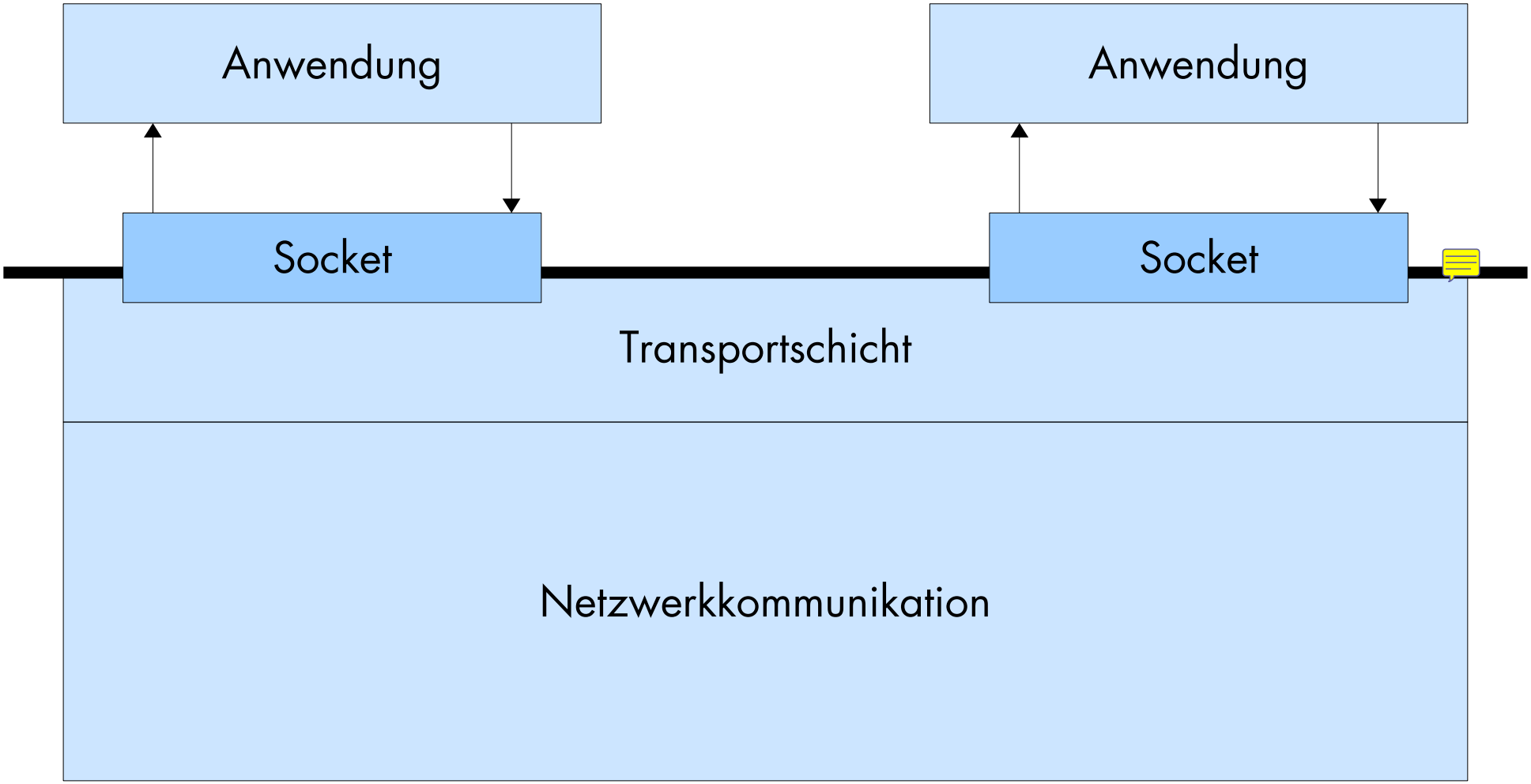
Referenzmodell

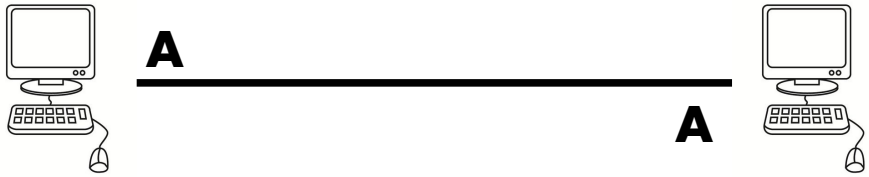


Transportschicht



Sockets


Anwendung
Anwendung
Socket
Socket
Transportschicht
Netzwerkkommunikation



Interprozesskommunikation mit TCP Sockets

Client | *Server*

Socket

bind()

listen()

Socket

connect()

accept()

Socket

listen()

send()

read()

send()

read()

close()

Übersicht

- Wiederholung
- TCP-Sockets
- UDP-Sockets
- Remote Method Invocation (RMI)

Java-Sockets

- Benötigte Klassen für Kommunikation im Paket: java.net
- Systemunabhängige Netzwerkkommunikation aufgrund der plattformunabhängig
- Implementierungen sowohl strombasiert(TCP), als auch paketorientiert(UDP) möglich

Interprozesskommunikation mit TCP Sockets

Client | Server

Socket

bind()

listen()

Socket

connect()

accept()

Socket

listen()

send()

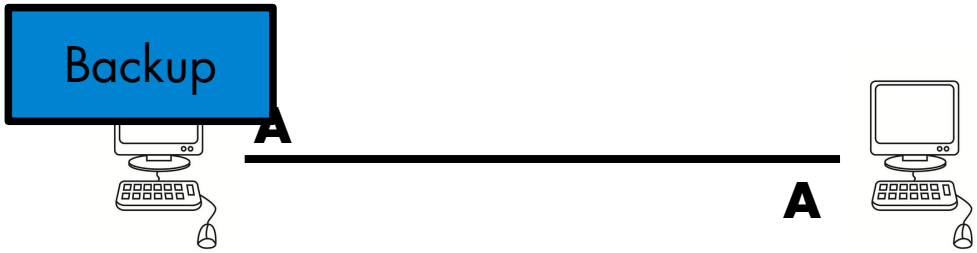
read()

send()

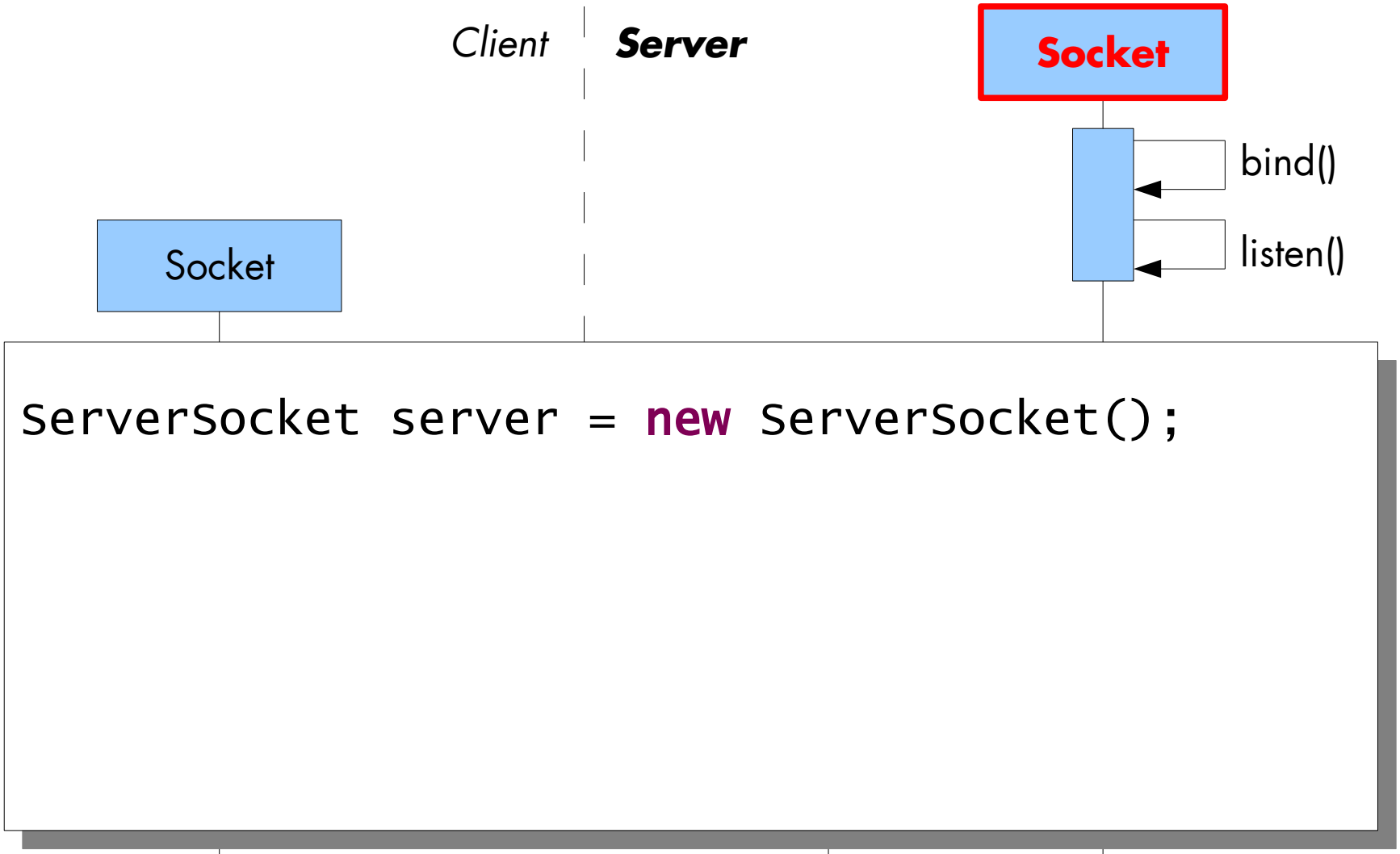
read()

close()

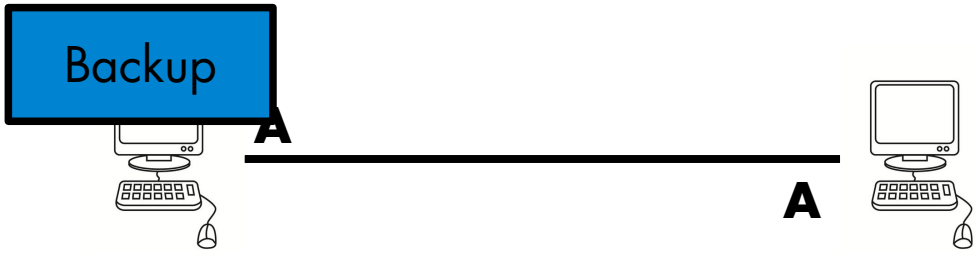
Snippet1



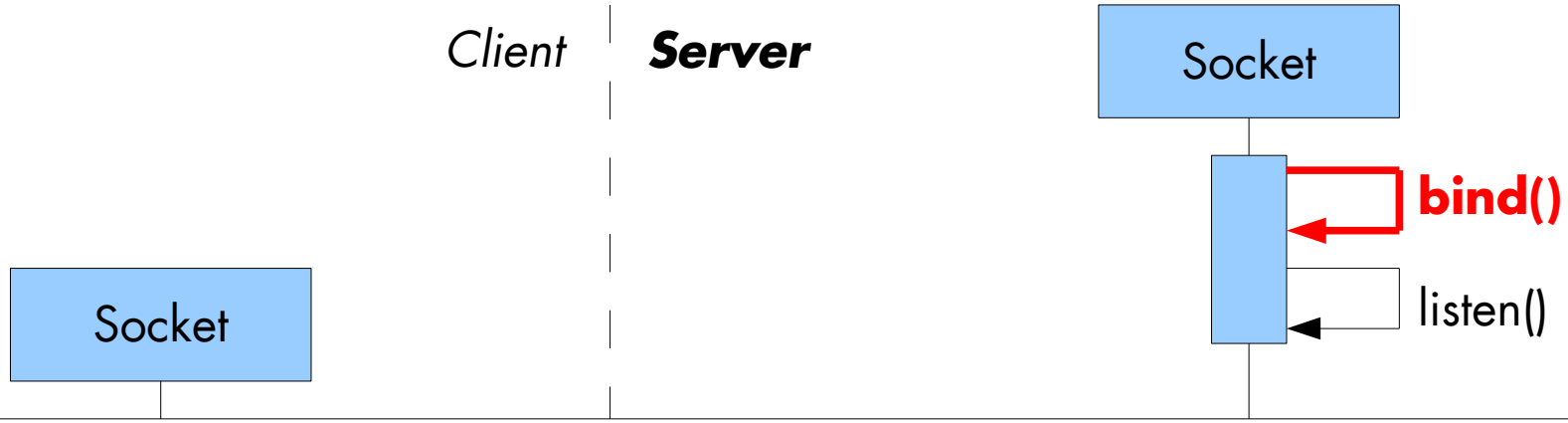
Interprozesskommunikation mit TCP Sockets



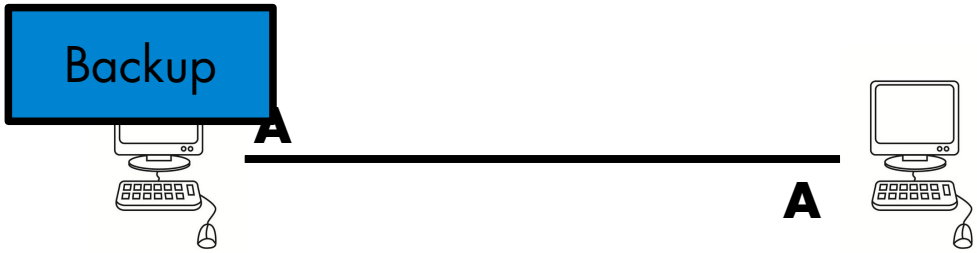

```
ServerSocket server = new ServerSocket();
```



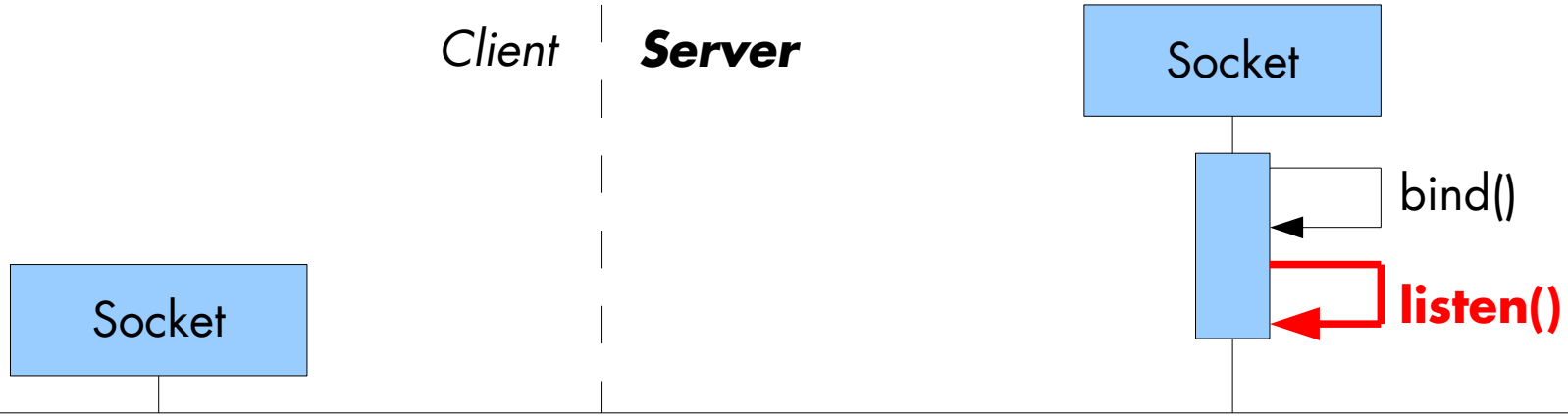
Interprozesskommunikation mit TCP Sockets



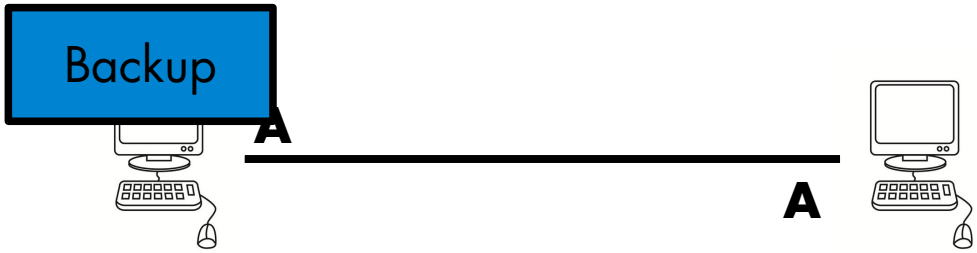

```
ServerSocket server = new ServerSocket();
server.bind(new InetSocketAddress());
```



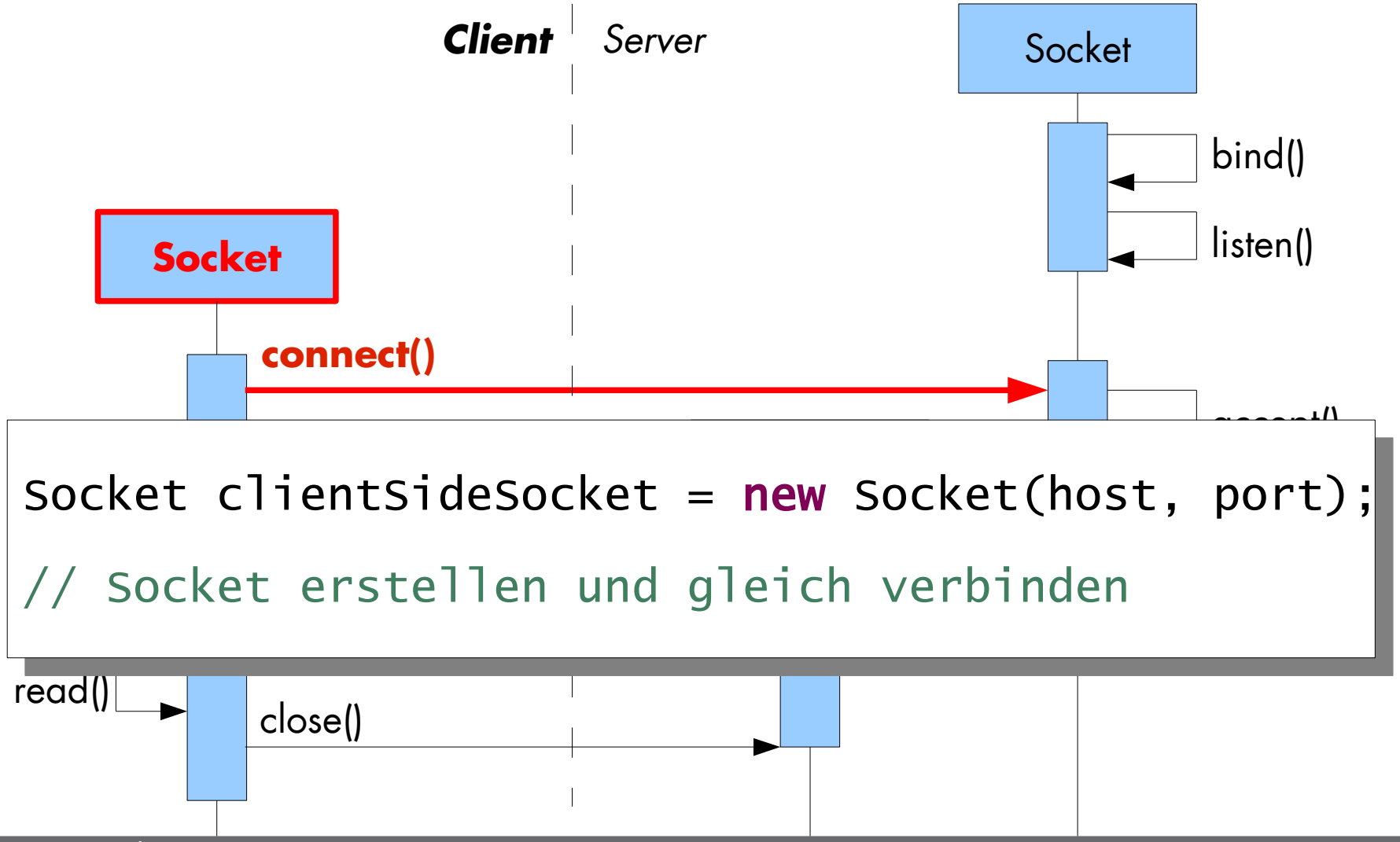
Interprozesskommunikation mit TCP Sockets



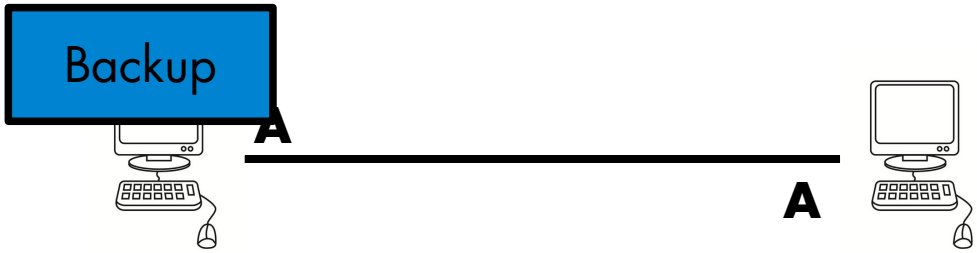

```
ServerSocket server = new ServerSocket();
server.bind(new InetSocketAddress());
Socket serverSideSocket = server.accept();
// Server wartet auf eingehende Verbindung
```



Interprozesskommunikation mit TCP Sockets

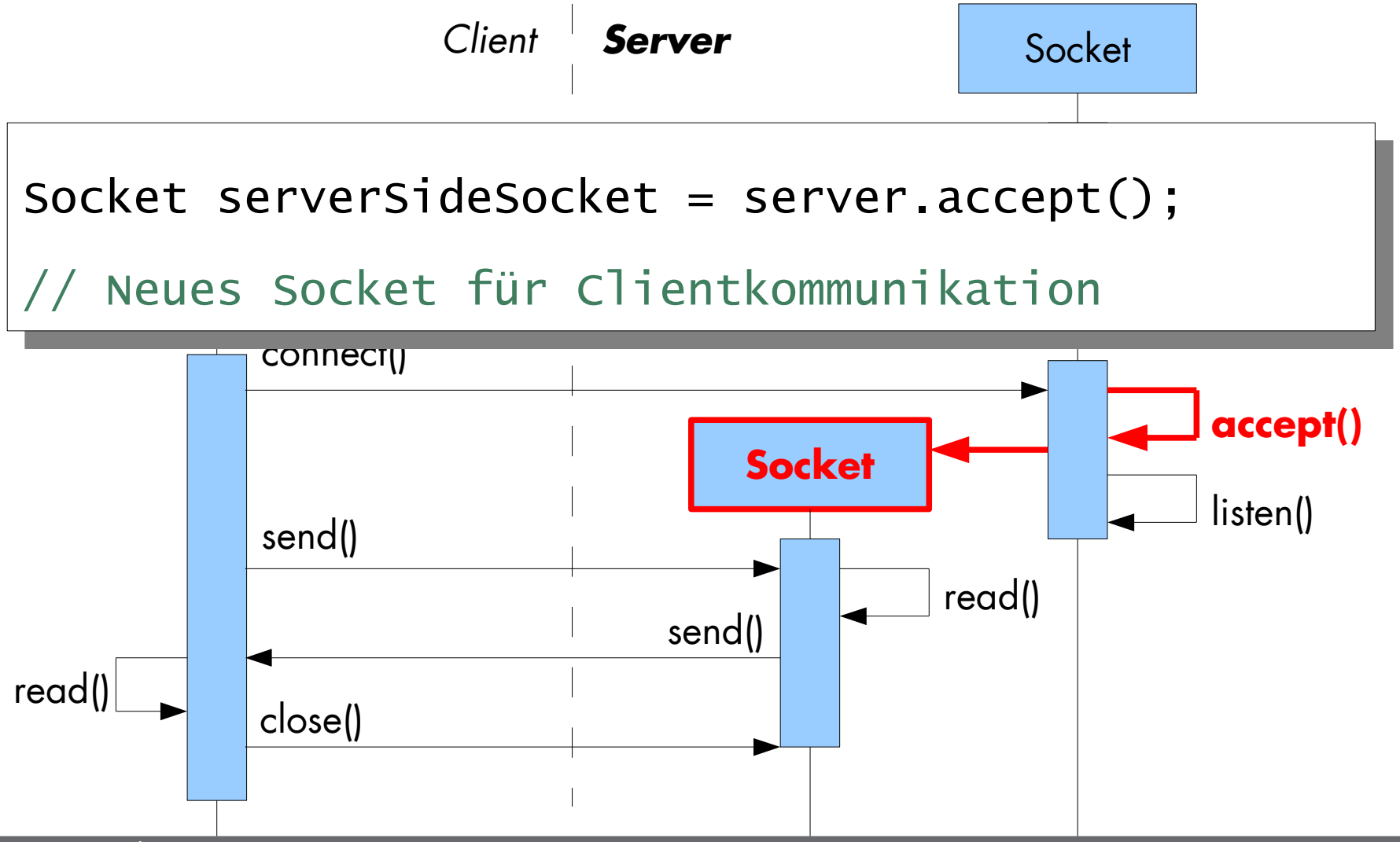


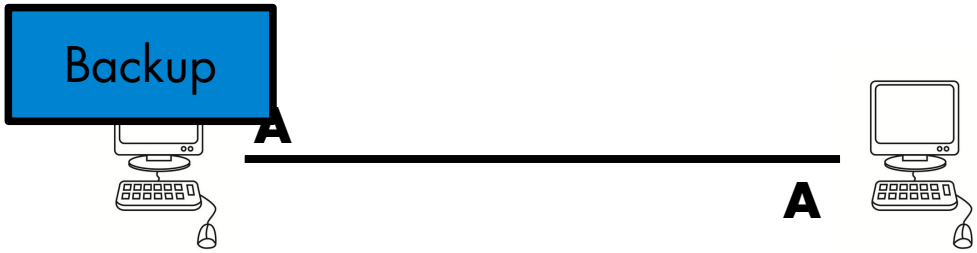

```
Socket clientSideSocket = new Socket(host, port);
// Socket erstellen und gleich verbinden
```



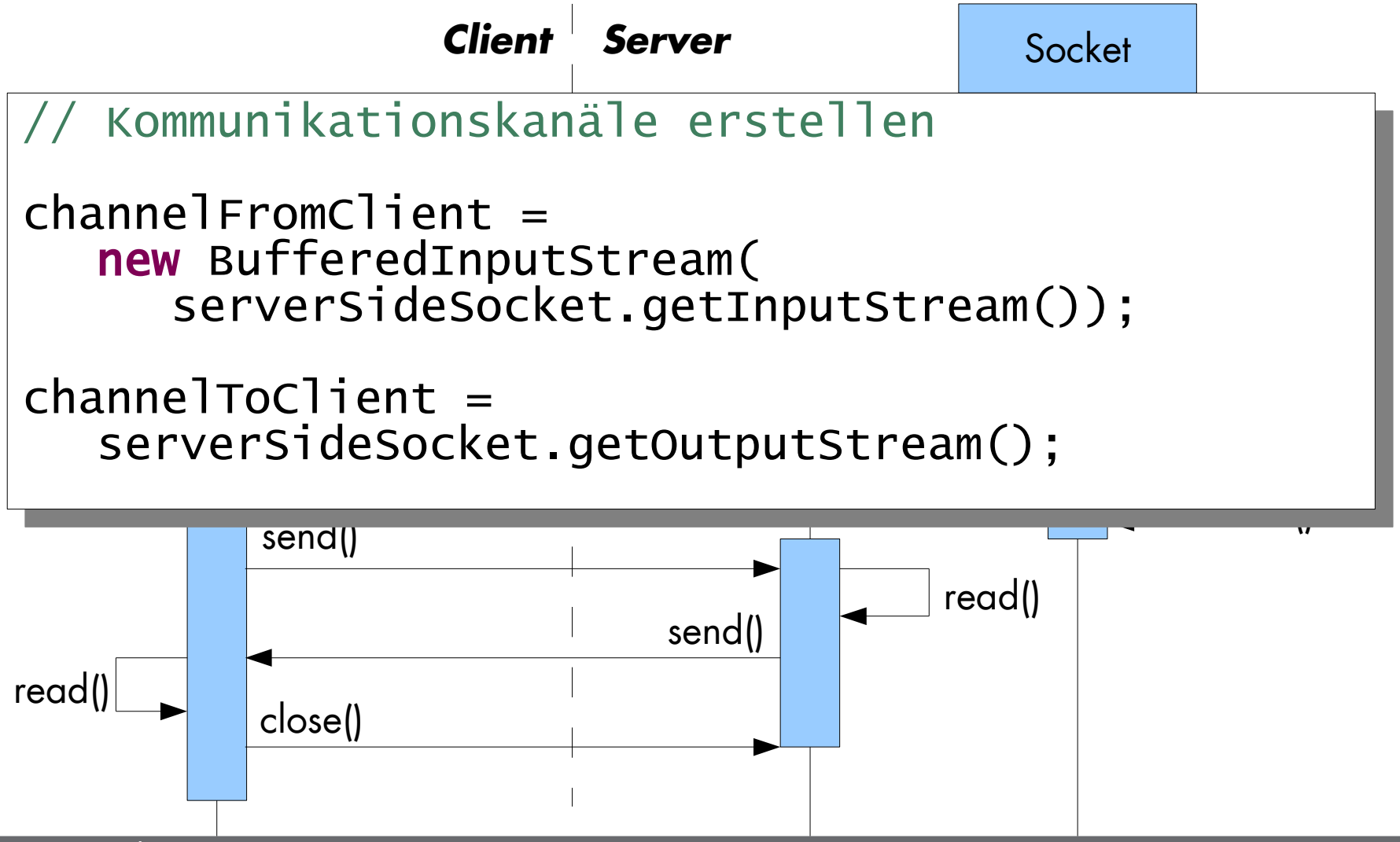
Interprozesskommunikation mit TCP Sockets

```
Socket serverSideSocket = server.accept();

// Neues Socket für Clientkommunikation
```

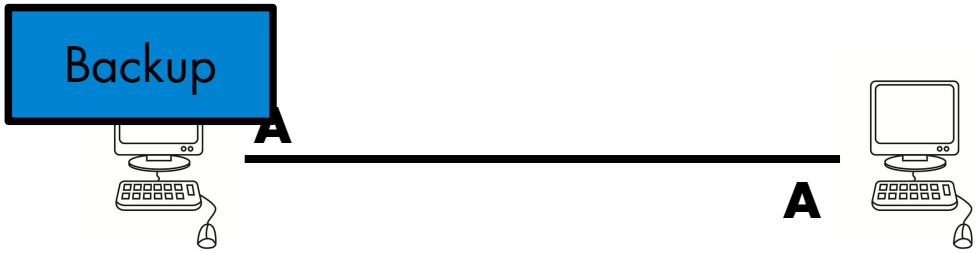




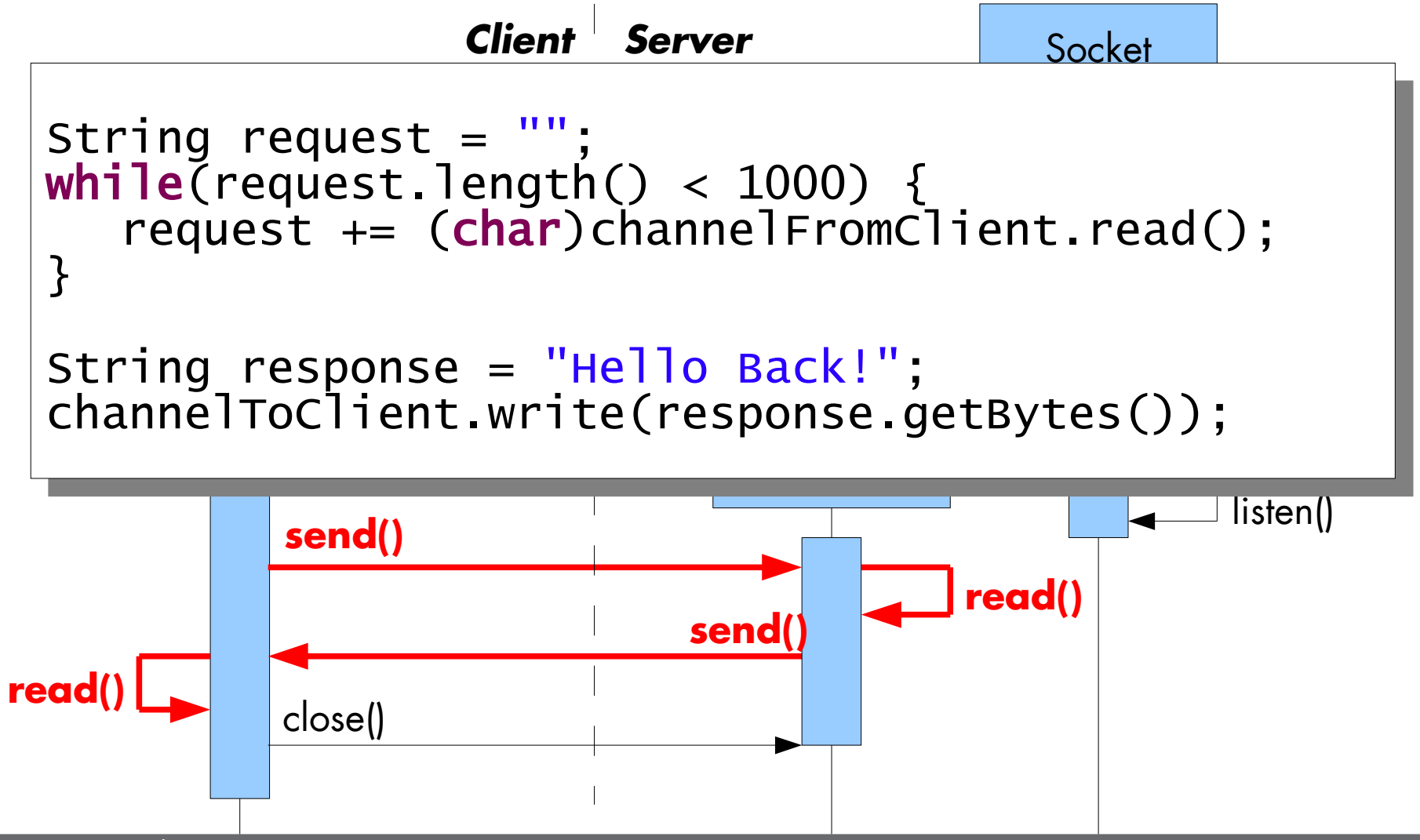
Interprozesskommunikation mit TCP Sockets



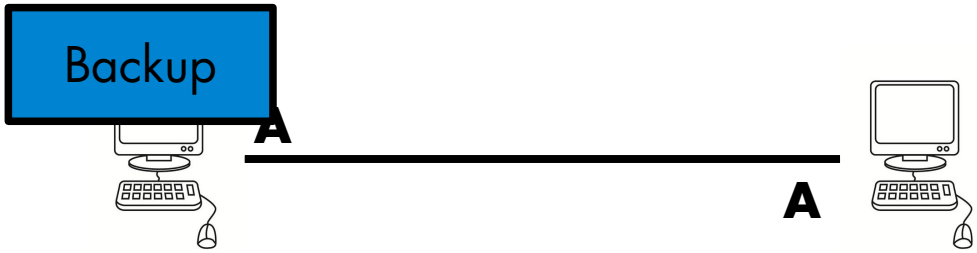

```
// Kommunikationskanäle erstellen

channelFromClient =
   new BufferedInputStream(
      serverSideSocket.getInputStream());

channelToClient =
   serverSideSocket.getOutputStream();
```



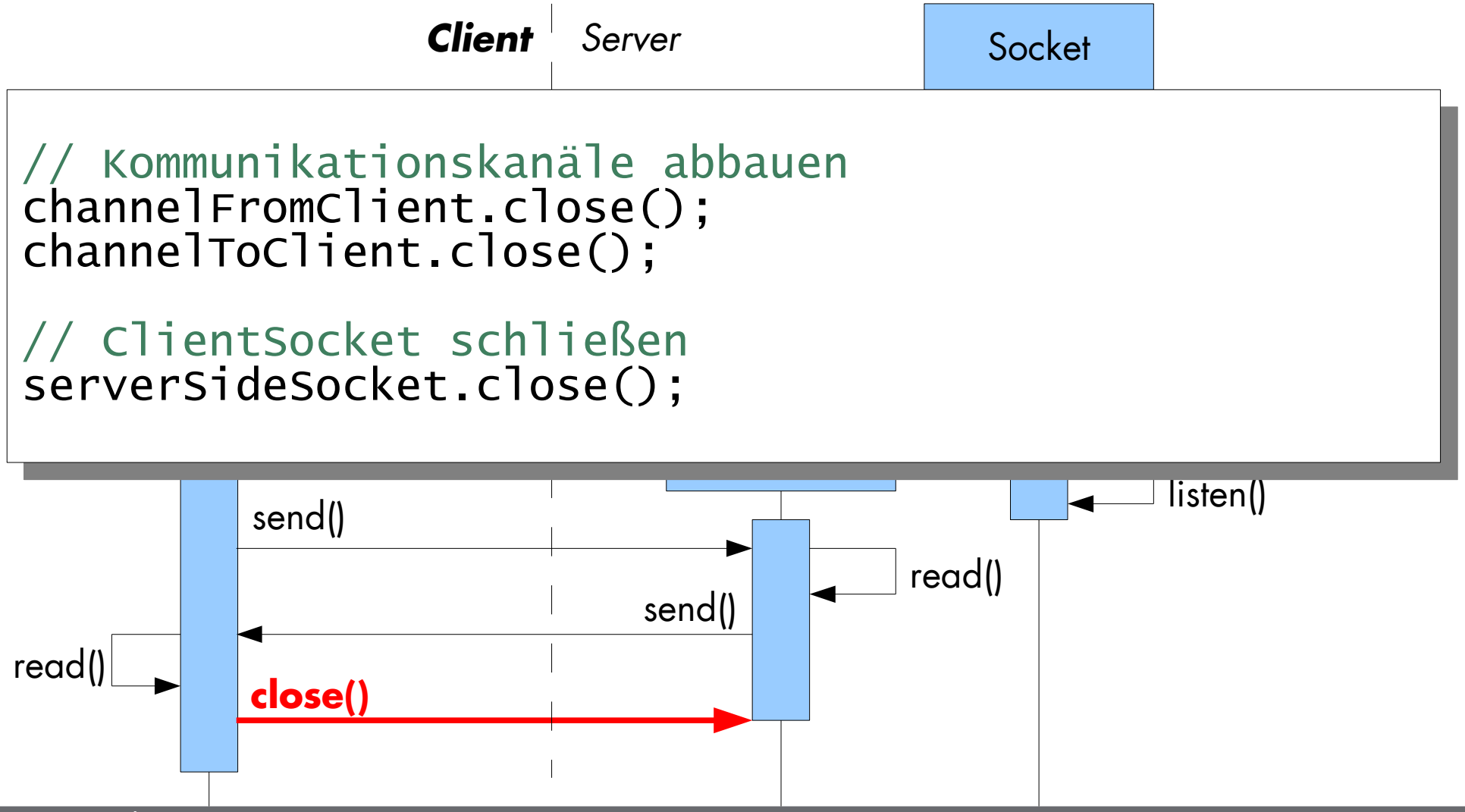
Interprozesskommunikation mit TCP Sockets




```
String request = "";
while(request.length() < 1000) {
   request += (char)channelFromClient.read();
}

String response = "Hello Back!";
channelToClient.write(response.getBytes());
```



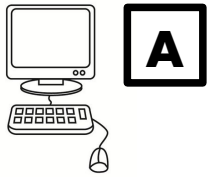
Interprozesskommunikation mit TCP Sockets

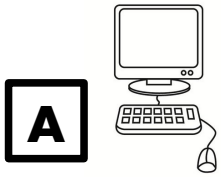



```
// Kommunikationskanäle abbauen
channelFromClient.close();
channelToClient.close();

// ClientSocket schließen
serverSideSocket.close();
```

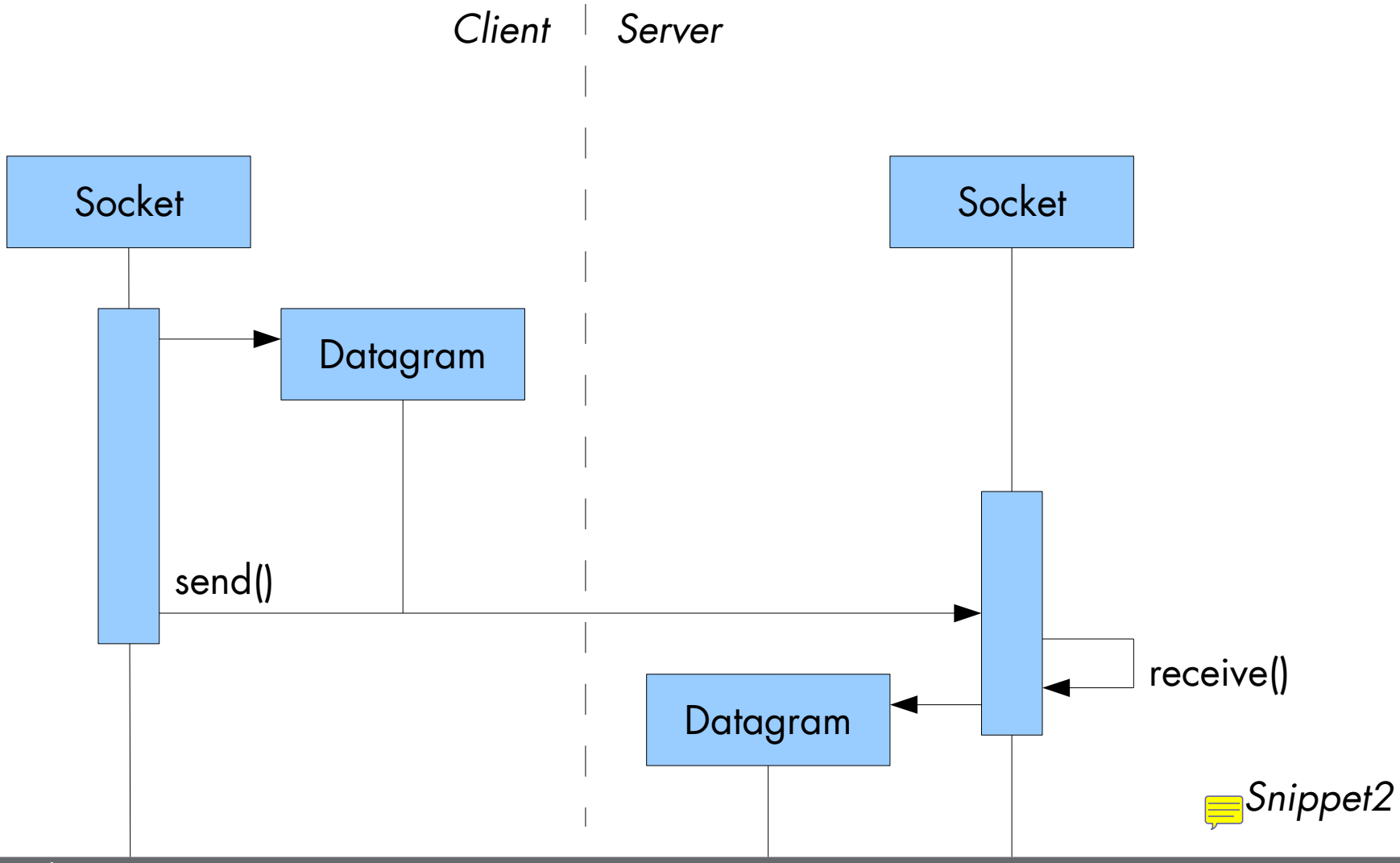
Übersicht

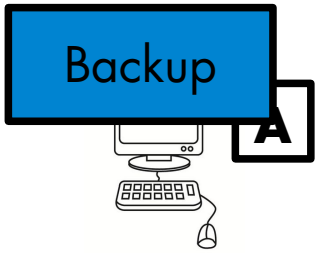
- Wiederholung
- TCP-Sockets
- UDP-Sockets
- Remote Method Invocation (RMI)

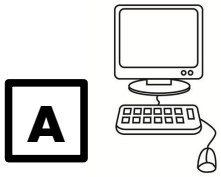




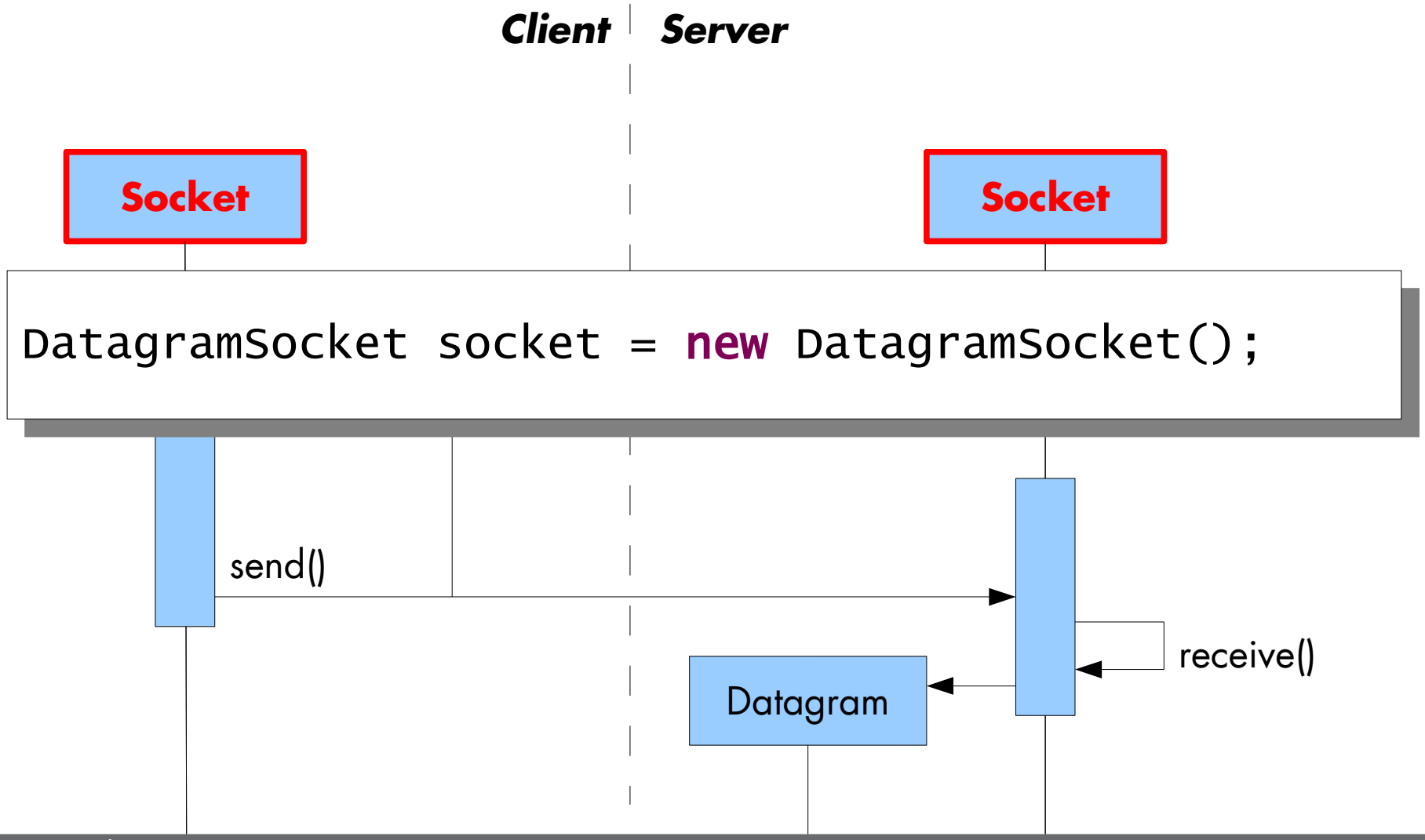
Interprozesskommunikation mit UDP Sockets

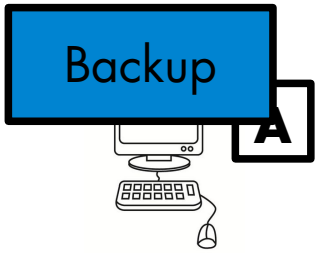


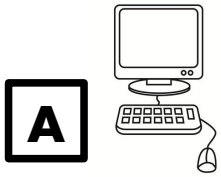




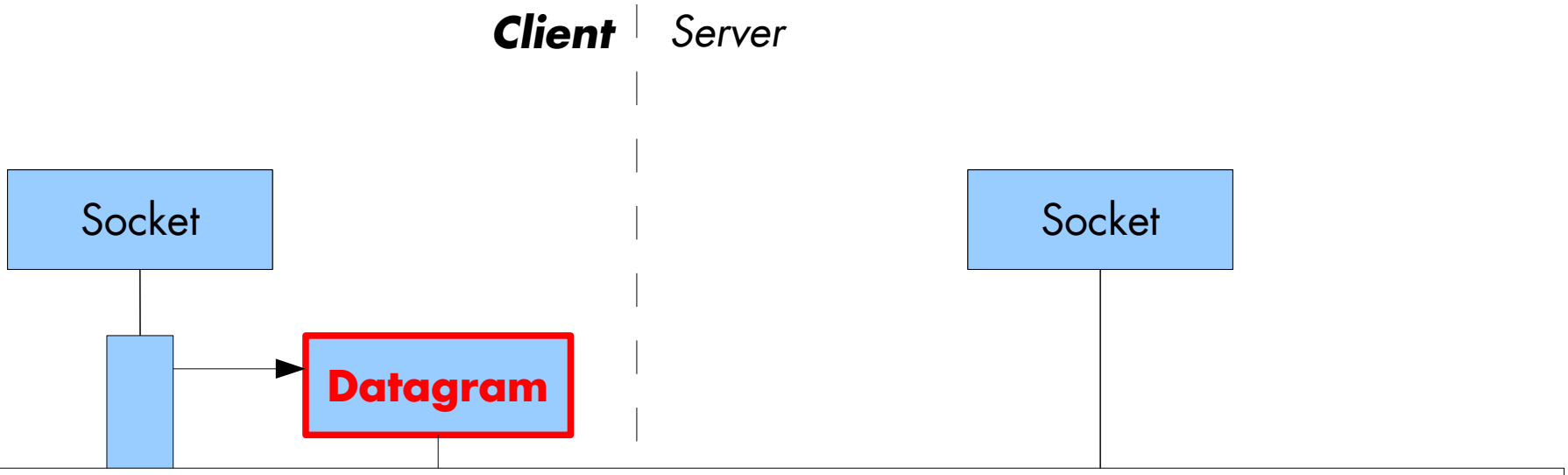
Interprozesskommunikation mit UDP Sockets



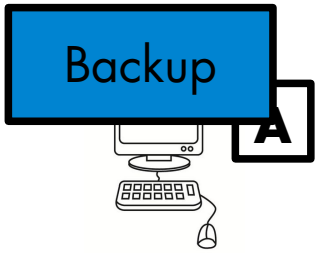


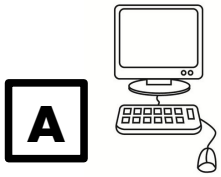


Interprozesskommunikation mit UDP Sockets




```
String host = "localhost";
InetAddress address = InetAddress.getByName(host);

String request = "AAAand byeZZZ";
DatagramPacket requestPacket =
   new DatagramPacket(request.getBytes(),
      request.getBytes().length, address, port);
```

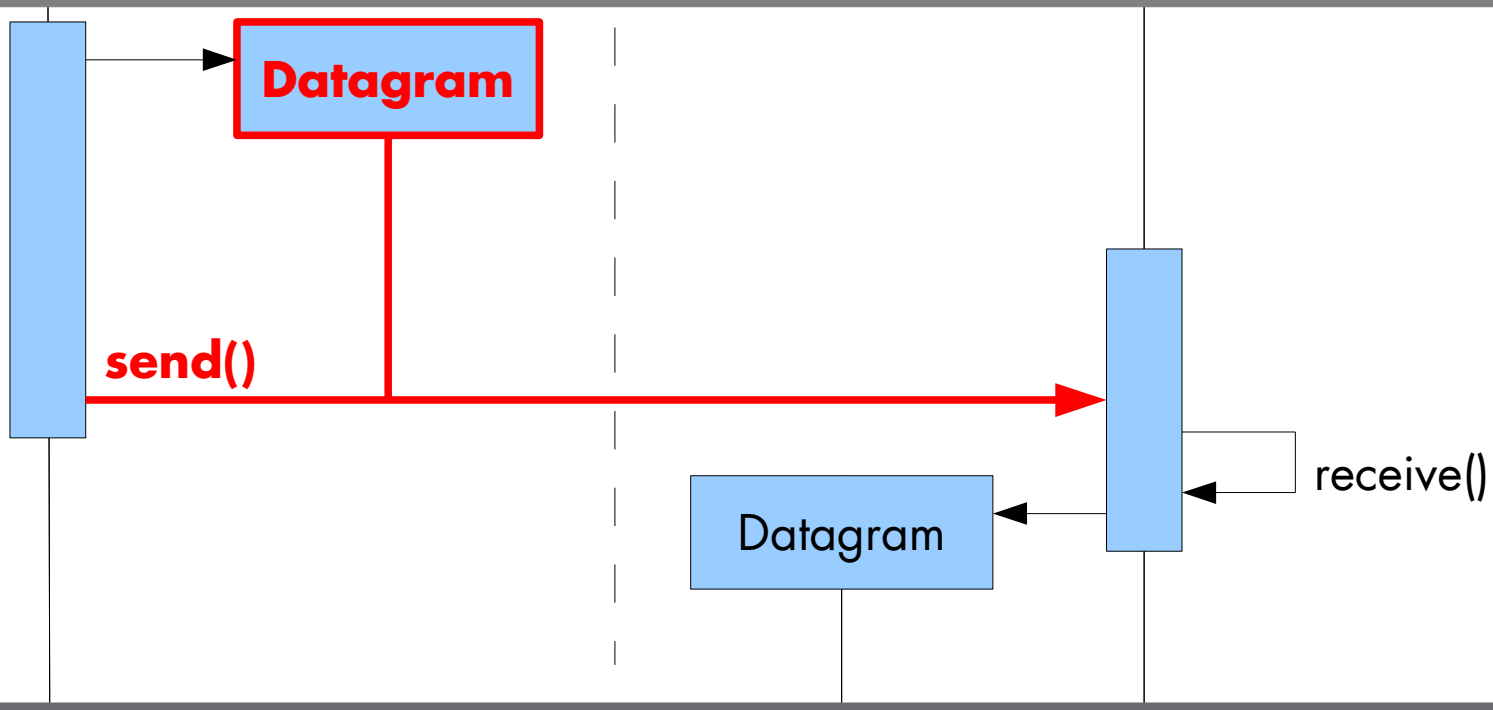


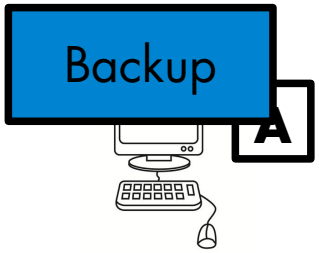


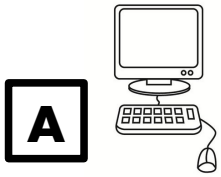
Interprozesskommunikation mit UDP Sockets

Client | *Server*

```
// Datagram senden

socket.send(requestPacket);
```



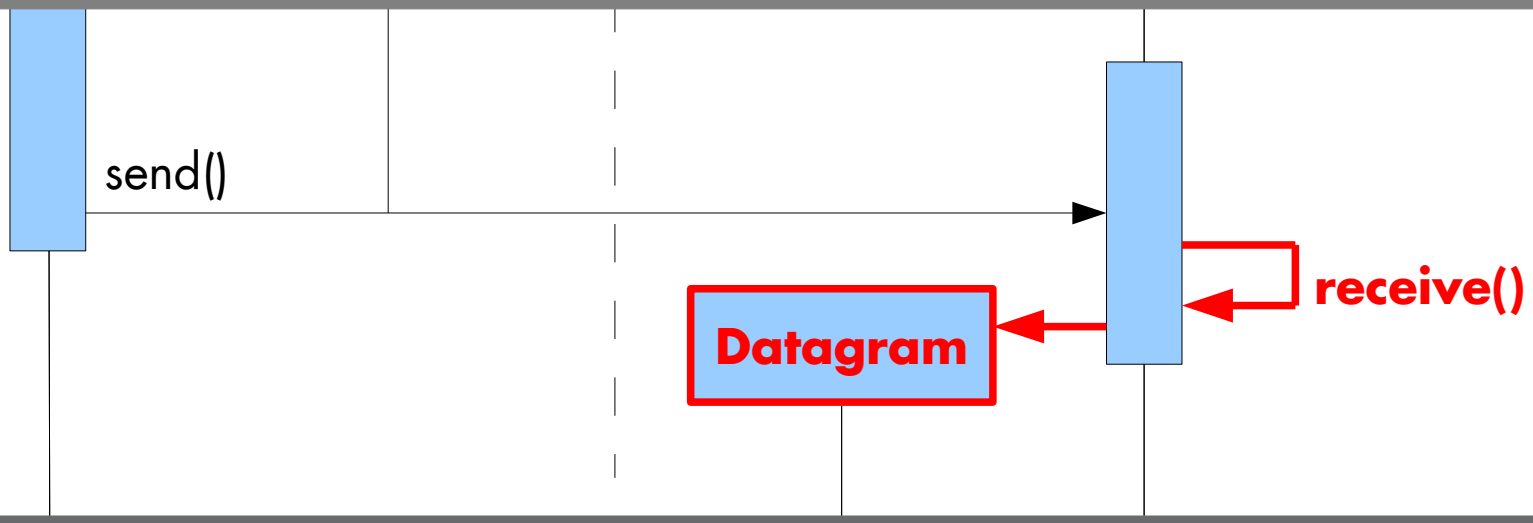




Interprozesskommunikation mit UDP Sockets

Client | ***Server***

```
// Antwort Datagram erstellen und empfangen

DatagramPacket responsePacket =
      new DatagramPacket(new byte[1000], 1000);

socket.receive(responsePacket);
```



Einschränkungen Socket-Kommunikation

- Beschränkung auf Zeichenkettenströme
- Aufwändige Implementierung bei komplexen Protokollen
- Weitergabe von geworfenen Ausnahmen sehr schwierig
- Probleme bei der Übergabe von Werten komplexer Datentypen
- Nutzung der Objektserialisierung

Beispiel für Aufruf einer Funktion zur Ermittlung von Primzahlen

- Für den Aufruf müssen übergeben werden
  - Ein Bezeichner der Aufgerufenen Funktion
  - Die Intervallgrenze von, als ganze Zahl
  - Die Intervallgrenze bis, als ganze Zahl
- Für die Rückgabe müssen übertragen werden
  - Eine beliebig lange Liste von einzelnen ganzen Zahlen
  - Eine mögliche leere Liste
- Die Funktion kann zudem sehr lange oder sehr kurz laufen
- Sinnvolle asynchrone Kommunikation
- Unterteilung einer großen Rückgabe-Liste in mehrere Blöcke

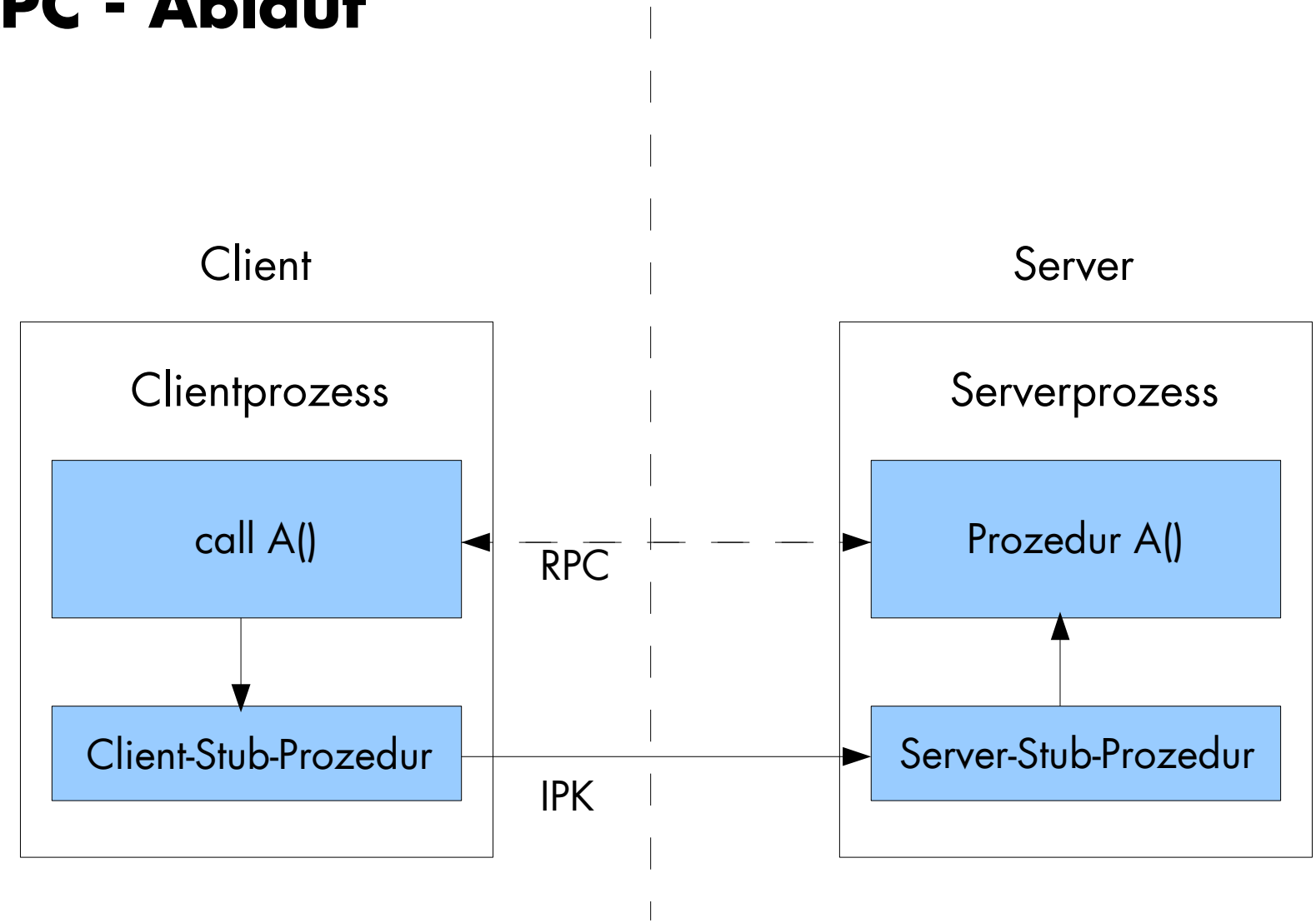
Übersicht

- Wiederholung
- TCP-Sockets
- UDP-Sockets
- Remote Method Invocation (RMI)

Aufruf von verteilten Prozeduren (RPC)

- Sockets sind niedrigste Form der Interprozesskommunikation(IPK)
- Interpretation der Nachrichten muss von Hand implementiert werden
- Entwicklung komplexerer Funktionen aufwändig und fehleranfällig
- IPK besser durch expliziten Aufruf von entfernten Prozeduren
- Konzept des Remote Procedure Call (RPC)
- Beruht auf lokalen Stellvertretern(Stubs)
- Simulieren normalen synchronen Prozeduraufruf
  - Leiten den Aufruf über Sockets an Server weiter
  - Server nimmt dies ebenfalls als normalen Prozeduraufruf war
  - Somit zuverlässige Syntax- und Aufrufbehandlung
  - Ggf. Typkonvertierungen und Formatanpassungen

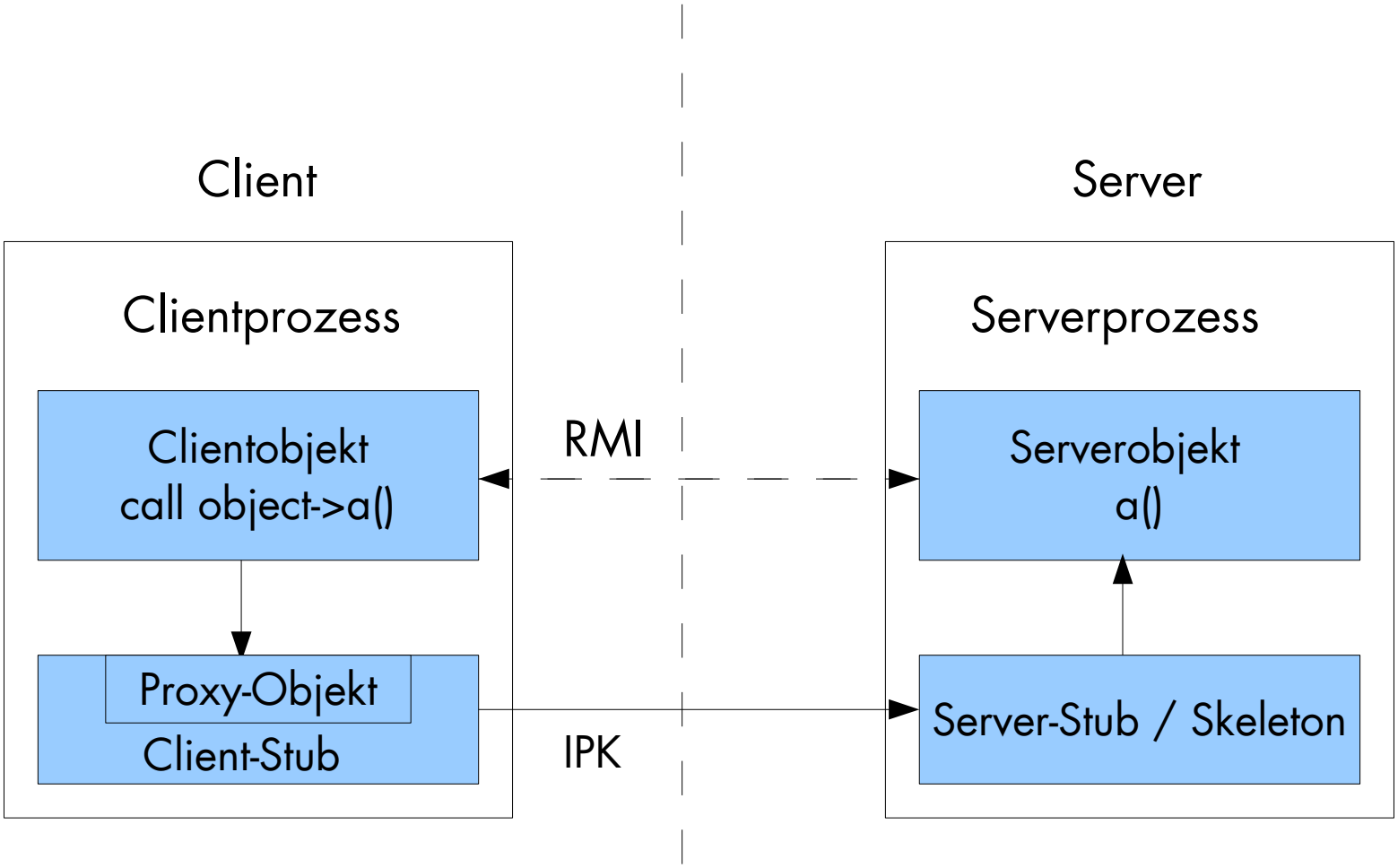
RPC - Ablauf


Client
Server
Clientprozess
Serverprozess
call A()
Prozedur A()
RPC
Client-Stub-Prozedur
Server-Stub-Prozedur
IPK

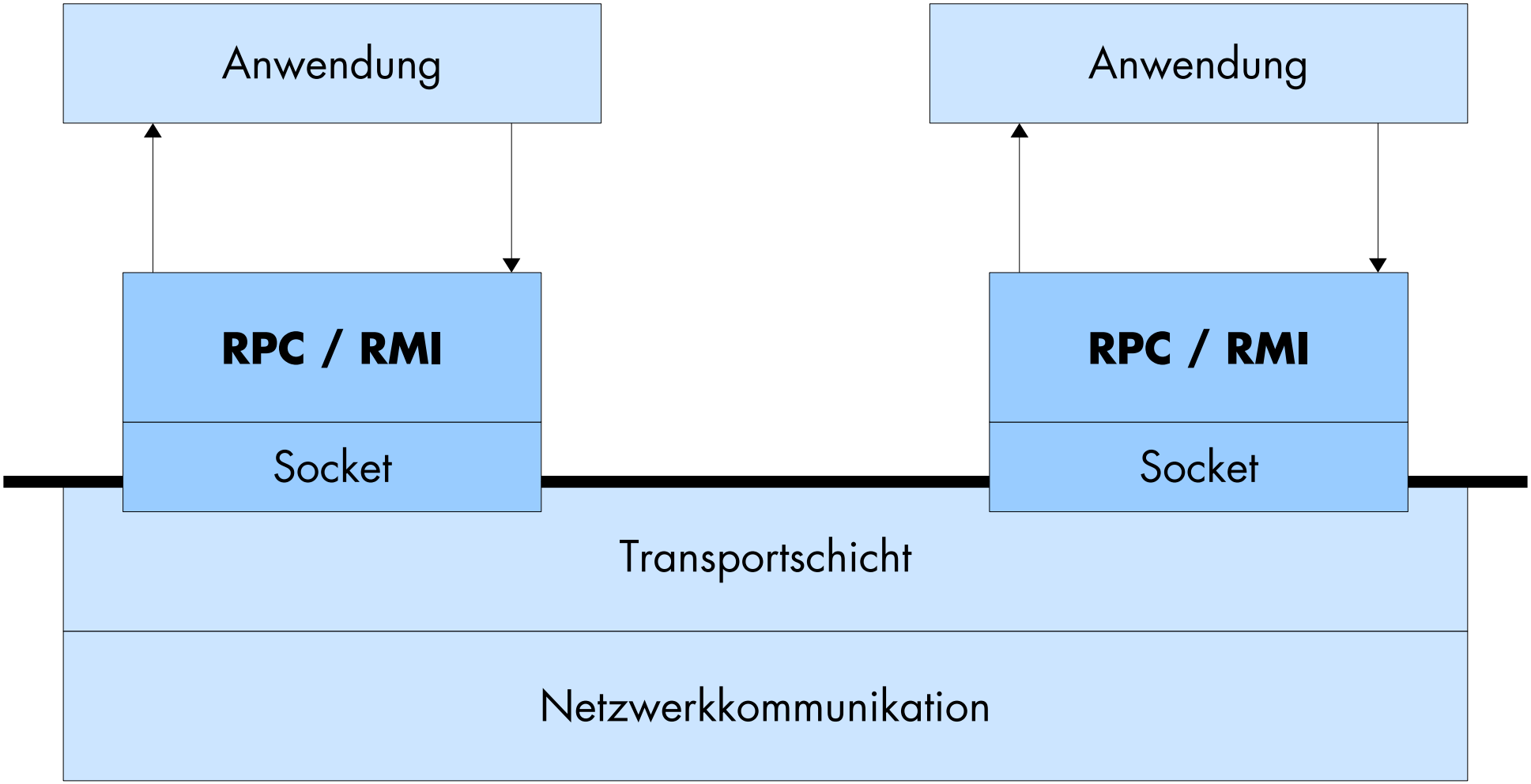
Remote Method Invocation (RMI)

- Objektorientierte Variante des *Remote Procedure Call* (RPC) in Java
- Konzept auf verteilten Objekten mit aufrufbaren Methoden
- Synchrone Kommunikation mit Aufruf und Ergebnisrückgabe
- Nutzt ebenfalls Stubs für transparente Kommunikation
- Zusätzlich auf Clientseite ein Stub-Objekt genannt Proxy
- Simuliert das entfernte Objekte
- Server-Stub wird auch als Skeleton bezeichnet

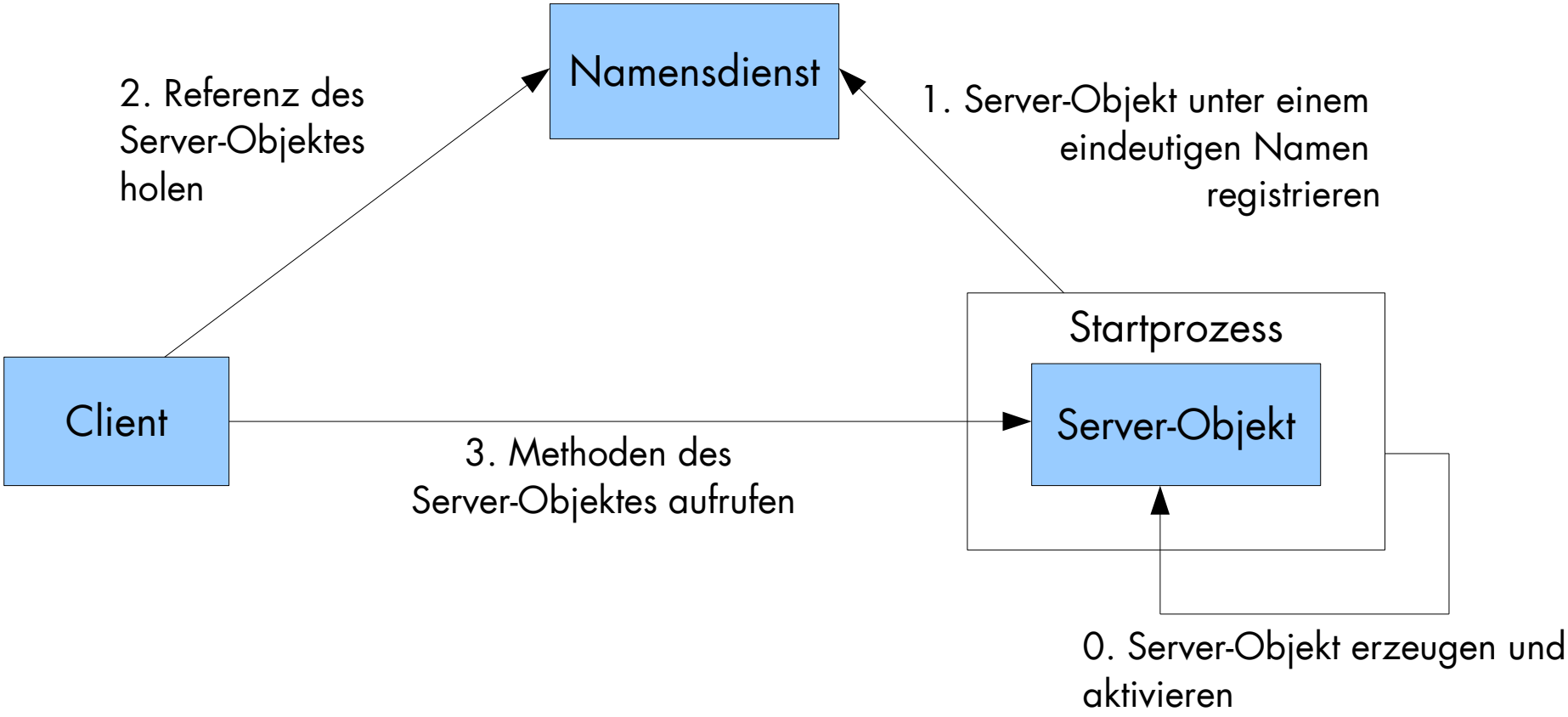
Remote Method Invocation (RMI)



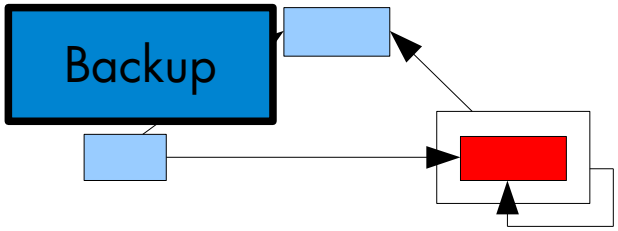
RPC/RMI und Sockets



Aufbau RMI



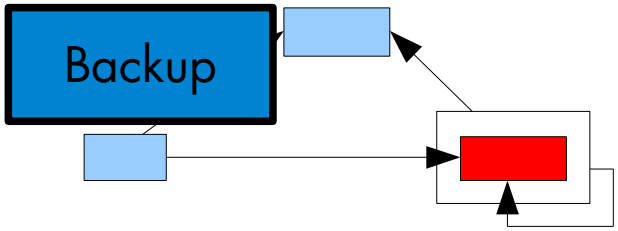
Snippet3



RMI – Serverseite - Remote-Interface

- Definition des Server-Dienstes als Schnittstelle
- Schnittstelle muss von `java.rmi.Remote` erben
- Somit ein sogenanntes *Remote-Interface*
- Methodensignaturen müssen `java.rmi.RemoteException` werfen

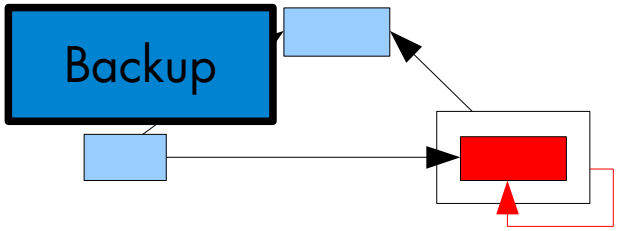
```
public interface MyRemoteInterface extends Remote{

    int myRemoteMethod() throws RemoteException;

}
```



RMI – Serverseite – Remote-Implementierung

- Verteilte Objekt implementiert Remote-Interface
- Kann zusätzlich auch lokale Implementierung enthalten

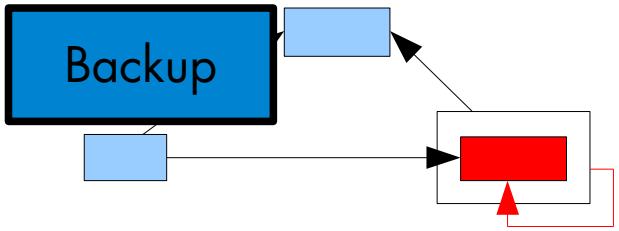
```
public class MyRemoteClass implements MyRemoteInterface

    // Methodenimplementierung
    int myRemoteMethode() throws RemoteException  {

    }

    ...
}
```



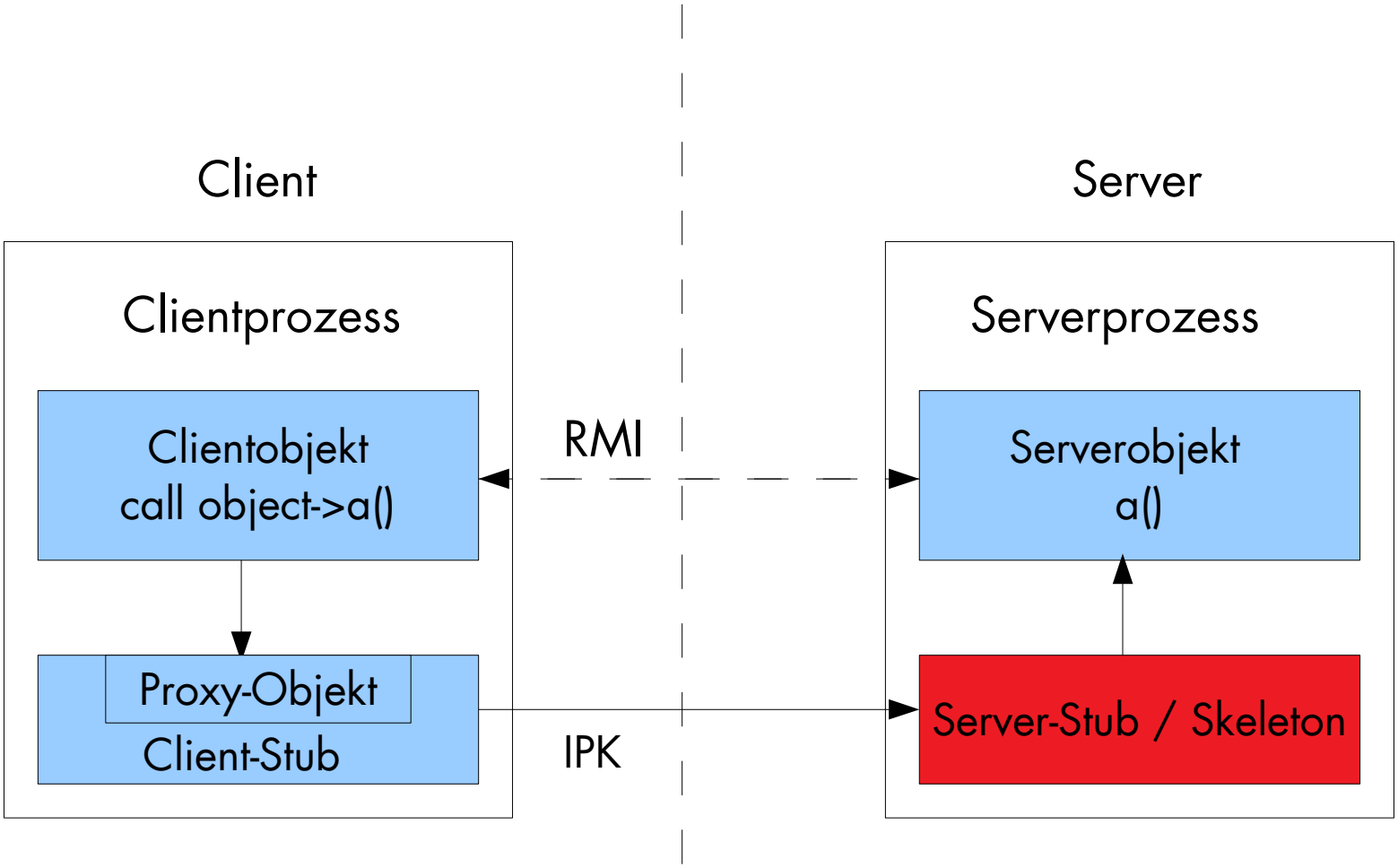
RMI – Serverseite – Verteiltes Objekt erzeugen

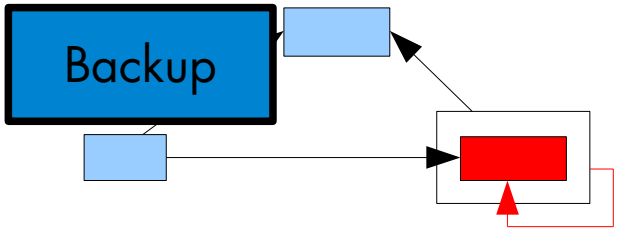
- Erzeugen des verteilten Objekts mit statischen Methode `UnicastRemoteObject.exportObject()`
- Erzeugt ein dynamisches Proxy-Objekt
- Per TypeCast Verwendung als Objekt

```
...
//Instanziierung
MyRemoteClass myRemoteObject = new MyRemoteClass();

...

//Erzeugung eines Proxy
MyRemoteInterface remoteInterface = (MyRemoteInterface)
      UnicastRemoteObject.exportObject(myRemoteObject,0);
...
```



Remote Method Invocation (RMI)

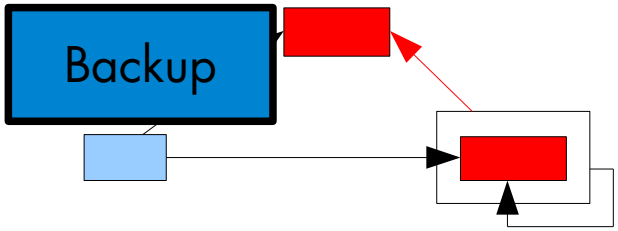




RMI – Serverseite – Verteiltes Objekt erzeugen

- Alternativ kann Serverobjekt auch von *UnicastRemoteObject erben*
- Automatischer Export neu erzeugter Objekte

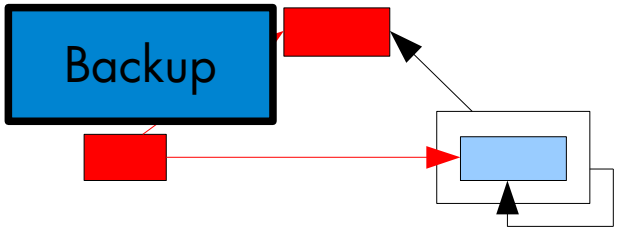
```
public class MyRemoteClass
    extends UnicastRemoteObject
    implements MyRemoteInterface {

    ...
    // Methodenimplementierung
    int myRemoteMethode() throws RemoteException  {

    }
    ...
}
```



RMI – Serverseite – Objekt registrieren

- Anbindung an die RMI - Registry
- Erzeugen einer neuen lokalen Registry
- Binden des verteilten Objekts an eine RMI-Registry

```
...
//Registry erzeugen auf Localhost, Port 1099
LocateRegistry.createRegistry(Registry.REGISTRY_PORT);

...
//Binden des dynamischen Proxy Objekts
Naming.rebind(„myRemoteService", myRemoteObject);

...
//Binden des von UnicastRemoteObject erbenden Objekts
Naming.rebind(„myRemoteService", new MyRemoteClass());
```



RMI - Clientseite

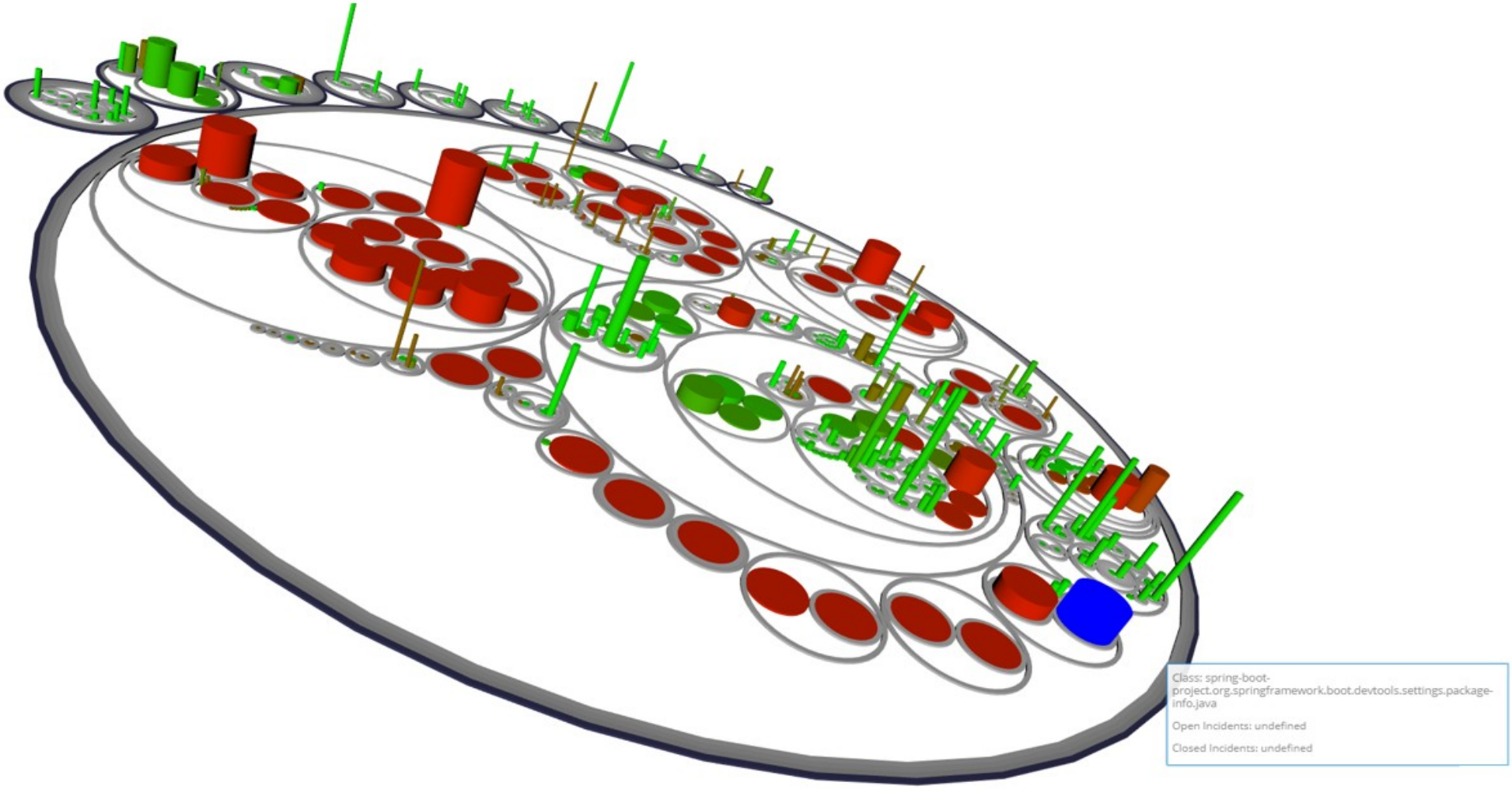
- Beschaffen einer Referenz zum entfernten Objekt
- Anfrage an einer Registry zum Beispiel mit
  `Naming.lookup("//<hostname>/<Servicebezeichnung>")`
- Casten der Objektreferenz zu RemoteInterface
- Entfernter Methodenaufruf auf Serverobjekt wie bei lokalem Objekt

```
...
// Ermittlung der Referenz auf das verteilte Objekt
MyRemoteInterface myRemoteObject = (MyRemoteInterface)
      Naming.lookup("//host/myRemoteService");
...

// Normaler Methodenaufruf auf Referenz des Serverobjekt
MyRemoteObject.myRemotheMethod();
```

Asynchrone Aufrufe mit RMI

- Nicht von Haus aus Unterstützt
- Main Feature ist der entfernte Methodenaufruf analog einem lokalen Aufruf, welcher per se synchron ist
- Asynchrone Kommunikation muss im Bedarfsfall selbst implementiert werden
  - Methodenaufruf per RMI auf Server
  - Servermethode startet eigenen Thread zur Verarbeitung der Anfrage
  - Thread ruft am Ende der Bearbeitung Methode auf Client per RMI
  - Clientmethode startet eigenen Thread zur Verarbeitung per Anfrage
- Alternativ asynchrone nachrichtenbasierte Kommunikation nutzen

Verteilte Garbage Collection

- Java RMI unterstützt Verteilte Garbage Collection
- Der Garbage-Collection-Algorithmus basiert auf Reference Counting
- Objektfreigabe wenn weder entfernte noch lokale Referenzen
- Client sendet regelmäßig eine Nachricht an den Server, wenn er noch mindestens eine Referenz auf das entfernte Objekt hält

Danke für Ihre Aufmerksamkeit